SW Engineering CSC648/848 Section 06

SpatialOS

Shared Spatial Operations and Localization Reliability

Team 02

Milestone 2

October 20, 2026

Member	Role
Daksh Kumar	Team Lead
Sumit Kumar	AI Master
Noah Bran	Back-end Lead
Calvin Mei	Front-end Lead
Ezeren Masangcay	GitHub Master
Tyler Duong	Scrum Master

Production site: sfsu-csc648-06-fall26-team02.vercel.app
GitHub: github.com/ezerenm/SFSU-csc648-06-Fall26-team02 (milestones/M2)

History

Version	Date	Change
M1 v1.0	Sep 29, 2026	Milestone 1 submitted
M1 rev.	Oct 8, 2026	Requirements split into small testable items with priorities and target milestones after the professor's feedback; answers to the professor's questions; phone platform changed to iPhone
M2 v1.0	Oct 20, 2026	Milestone 2 submitted: data definitions V2, functional requirements V2, UI mockups and UX flows, architecture/DB/APIs, UML, risks, project management; vertical prototype deployed

1. Data Definitions V2

This section expands the M1 data definitions into full field-level definitions. The same names are used in the database, the API JSON, the TypeScript types (`lib/spatial/contracts.ts`), the UML in Section 5 and the UI. Each entity is tagged **[M2]** if it is built and used in the M2 prototype, **[M2 partial]** if only part of it is built, or **[Planned]** if it is built in M3 to M5. Planned entities are still fully defined here because they are part of the full design.

1.1 Conventions

Convention	Definition
Naming	`snake_case` in SQL and API JSON. Table names are plural (`graph_nodes`); entity names in the UML are singular (GraphNode).
Terms	A path segment is a walkable connection between two graph nodes. Section 2 uses the same term; the M1 requirements called it an *edge*.
IDs	UUID. Every reference points to an existing record.
Time	`timestamptz` (UTC), ISO 8601 in the API. Displayed in local time.
Units	Positions and distances in meters, latency in milliseconds, heading in degrees. Distances are stored to the millimeter and shown with one decimal (5.1 m).
Missing data	`null`, never 0. A missing latency, pose or error is never stored as zero.
Enums	Fixed value lists; unknown values are rejected by the database and the API. Exception: pose `data_origin` is not taken from the client as is. The server sets it to `synthetic` for the fake-test device or when the client sends `synthetic`, and to `real` otherwise (1.10).
Text limits	Names and labels 120 characters; descriptions and reasons 1,000 characters.
Map scope	Every spatial record belongs to one map version; linked records must share it and its graph revision.

Shared values

Value	Definition
`position`	`{x_m, y_m, z_m}` in world meters.
`orientation`	Quaternion `{x, y, z, w}`. Expected to be normalized; in M2 the API checks only that the four values are numbers, and normalization checks follow in M3.
`world_frame`	Object `{frame_id, origin_description, handedness: "right", up_axis: "y", units: "meters"}`. Right-handed, Y up, X and Z horizontal. Stored on each map version.
`provider_pose`	Raw phone pose as reported by the Niantic SDK in Unity: `{position: {x, y, z}, rotation: {x, y, z, w}}`.
`route_step`	`{segment_id, from_node_id, to_node_id}` (UUIDs); gives the travel direction explicitly.

How coordinates line up

Data	Frame	How it gets into the world frame
World frame (HSS 349)	VPS map frame of *SFSU HSS 349 v2*	This is the world frame of the HSS 349 map version.
Scanned 3D model (GLB)	Same VPS map frame	Exported from the VPS map, so no conversion is needed (identity).
Graph nodes, destinations	World frame	Recorded on site with the iPhone while localized on the same VPS map. `y_m = 0` is phone height; the viewer draws the graph at the floor height measured in the scan (−1.29 m).
Test points	World frame	The location is picked on the model. The reference position used to measure accuracy is measured independently on site, not taken from the model (1.8).

Data	Frame	How it gets into the world frame
iPhone pose from VPS	Unity coordinates (left-handed) of the same VPS map	Converted by the server: world = (x, y, −z). From M3, a map whose model and VPS frame differ uses a verified `frame_transforms` row (1.12).

1.2 Users, roles and privileges

[Planned] Application roles. The roles from M1 are kept. Accounts and passwords are managed by Supabase Auth and never stored in application tables. Permissions are enforced by the API (RoleGuard, Section 5) and by database row-level security, never by the UI alone.

Role	Can do
`visitor`	View the environment and destinations; navigate; see own session, pose and route.
`developer`	Visitor abilities, plus run localization attempts and labeled simulation or replay, and view site evaluation results.
`operator`	Visitor abilities, plus edit destinations and path segments, open or close segments, set robot access, and review agent reports.
`administrator`	Manage accounts, site memberships, site configuration and map publication.

[M2] Access in the prototype. M2 has no sign-in yet. Access is limited as follows:

Actor	Can do in M2
Anyone with the site URL	Through the API: read the published public demo maps (`sites.is_public_demo`), their graph, destinations and synthetic attempts, and request routes. Directly from Supabase (anon key): read pose samples. No writes. The current pose read policy allows reading every pose; limiting it to public demo maps is planned for M3.
iPhone app with the device token	Add pose samples through POST /api/poses. The token is a server-side environment variable (`DEVICE_TOKEN`), never stored in a table.
Team, TA and instructor (Supabase project members)	Full table access in the Supabase dashboard, for seeding and migrations.

user_profiles [Planned]

Field	Type	Notes
`user_id`	UUID	Same as the Supabase Auth user ID
`display_name`	text	Shown in the app, max 120 characters
`created_at`	timestamp	When the profile was created

site_memberships [Planned]

Field	Type	Notes
`membership_id`	UUID	Primary key
`user_id`	UUID	Links to `user_profiles`
`site_id`	UUID	Site the role applies to
`role`	enum	`visitor`, `developer`, `operator`, `administrator`; one active role per user per site
`is_active`	boolean	False = access removed
`created_at`	timestamp	When the membership was granted

1.3 Site [M2]

`sites`: a physical area. M2 has two published public demo sites:

- **SFSU HSS 349**: the real scanned classroom used for the 3D viewer, routing and live position.
- **SYNTHETIC M2 demo**: invented coordinates and labeled sample attempts for the list and search prototype (`is_test_site = true`). It is not a campus scan.

Field	Type	Notes
`site_id`	UUID	Primary key
`name`	text	e.g. "SFSU HSS 349"
`description`	text	Area covered
`is_test_site`	boolean	`true` for seeded test data
`is_public_demo`	boolean	`true` = readable without sign-in in M2
`active_map_version_id`	UUID, nullable	Published map used by default
`created_at`, `updated_at`	timestamp	Set by the database on insert and update

1.4 Map version [M2]

`map_versions`: one captured version of a site.

Field	Type	Notes
`map_version_id`	UUID	Primary key
`site_id`	UUID	Owning site
`version_label`	text	Unique per site, e.g. "HSS349-v1"
`publication_state`	enum	`draft`, `published`, `retired`; published requires `published_at`
`world_frame`	object	See 1.1
`visual_asset_id`	UUID, nullable	Displayed 3D model (1.12). Null in M2; the viewer loads the model file directly.
`vps_map_id`	text, nullable	Niantic localization map ID; separate from the visual asset
`vps_status`	enum	`not_configured`, `draft`, `production`, `unavailable`; draft and production require `vps_map_id`
`current_graph_revision`	integer	Topology version
`current_path_state_revision`	integer	Open/closed and robot-access version
`captured_at`, `published_at`	timestamp, nullable	When the area was scanned and when this version was published; published requires `published_at`

The visual model is what users see; the VPS map is what the phone localizes against. They are stored separately. For HSS 349 they share one frame (1.1); other maps are aligned through a frame transform from M3 (1.12).

1.5 Graph nodes [M2]

`graph_nodes`: waypoints that path segments and destinations connect to.

Field	Type	Notes
`node_id`	UUID	Primary key; referenced by path segments and destinations
`map_version_id`	UUID	Map version the node belongs to

Field	Type	Notes
`graph_revision`	integer	Graph revision the node belongs to; must match the map's current revision to be shown
`label`	text	e.g. "N1 Door"
`position`	position	World meters

HSS 349 has four nodes: N1 Door, N2 Back corner (door side), N3 Back corner (far) and N4 Front of room. The UI shows each node by the name of its destination (1.7).

1.6 Path segments [M2]

`path_segments`: walkable connections between two nodes.

Field	Type	Notes
`segment_id`	UUID	Primary key; referenced by route steps
`map_version_id`, `graph_revision`	UUID, integer	Same revision as both endpoints
`label`	text	Searchable, e.g. "Front to Door"
`start_node_id`, `end_node_id`	UUID	Different nodes
`direction`	enum	`bidirectional`, `start_to_end`
`distance_m`	number	Positive; must match the geometry length within 0.01 m
`geometry`	array of position	2 to 1,000 points; first and last point equal the start and end node positions
`availability`	enum	`open`, `closed`
`robot_access`	enum	`allowed`, `restricted`
`last_state_revision`	integer	Path-state revision of the last open/closed or robot-access change
`updated_at`	timestamp	Set by the database on every update

A human route may use any **open** segment. A robot route may use only segments that are **open and robot-allowed**. Reopening a segment keeps its robot access rule. The database rejects a segment whose geometry or distance does not match its nodes.

1.7 Destinations [M2]

`destinations`: named places users can navigate to. HSS 349 has four, one at each node: **Door, Back Corner (Door Side), Back Corner (Far Side)** and **Front of Room**. The synthetic demo map has three (Room 103, Lobby, East room).

Field	Type	Notes
`destination_id`	UUID	Primary key; sent in route requests
`site_id`, `map_version_id`	UUID	Site and map version the destination belongs to; must match its node
`graph_revision`	integer	Same revision as its node
`node_id`	UUID	Destination node in the same graph
`name`	text	Searchable; shown in the Start and Destination lists
`description`	text	Short explanation shown with the name, e.g. "Room entrance"
`is_active`	boolean	Offered for new navigation

Field	Type	Notes
updated_at	timestamp	Set by the database on every update

1.8 Test points [M2]

test_points: fixed physical camera positions used to evaluate localization. The full design has five evaluation points per site; points used for alignment are kept separate. In M2 the five test points belong to the synthetic demo map (is_synthetic = true) and are used only for the attempt search.

Field	Type	Notes
test_point_id	UUID	Primary key; used as a search filter
map_version_id	UUID	Map version the test point belongs to
label	text	Searchable, unique per map, e.g. "TP3 lobby"
purpose	enum	evaluation, alignment
reference_position	position, nullable	Camera position measured independently on site (not taken from the model)
reference_method	text, nullable	How it was measured
reference_uncertainty_m	number, nullable	Never assumed 0
reference_is_verified	boolean	Only verified points count toward error; requires a real point with position, method and uncertainty
is_synthetic	boolean	true for seeded test data

1.9 Localization attempts [M2]

localization_attempts: one record per started localization attempt, including every failure and timeout. **Success** means a valid pose for the intended map within 30 seconds; accuracy is measured separately.

Field	Type	Notes
attempt_id	UUID	Resending the same attempt keeps its ID; a new attempt gets a new ID
session_id	UUID, nullable	Links to a session from M3; null in M2
map_version_id, test_point_id	UUID	Map and test point of the attempt; the test point must belong to the map
device_id	text	Pseudonymous device label
sdk_version	text, nullable	Required for live attempts; null for synthetic data
mode	enum	live, simulation, replay
data_origin	enum	live_sdk, synthetic, recorded (pairs with mode: live/live_sdk, simulation/synthetic, replay/recorded)
condition_label	text	e.g. "daylight", "evening"
camera_direction	text	e.g. "facing north"
started_at, ended_at	timestamp	ended_at is null while in progress
outcome	enum	in_progress, success, timeout, tracking_loss, network_error, map_mismatch, cancelled, other_failure
latency_ms	number, nullable	Success only, 0 to 30,000
failure_reason	text, nullable	Required for failures

Field	Type	Notes
valid_pose_sample_id	UUID, nullable	Required for a live success from M3; null in M2
horizontal_error_m	number, nullable	X/Z distance to a verified reference; null in M2
error_is_valid	boolean	False for synthetic data and in M2
received_at	timestamp	Server receipt time

Horizontal error = $\sqrt{((pose.x - ref.x)^2 + (pose.z - ref.z)^2)}$. In M2 all attempts are labeled synthetic samples (`mode = simulation`), and the database itself keeps the M3 fields empty, so no sample can be mistaken for a measured result. Synthetic and replayed attempts never count as live evidence.

1.10 Pose samples [M2 partial] and sessions [Planned]

A **pose sample** is one position report from the phone. It is built in M2: the iPhone app sends one every 0.5 s and the /live page shows the latest one as a moving dot. A **session** groups one device's live, simulation or replay data from M3.

pose_samples [M2 partial]

Field	Type	Notes
sample_id	UUID	Created by the phone; sending the same ID again stores nothing new
map_version_id	UUID	Map the pose belongs to; must be published
attempt_id	UUID, nullable	Localization attempt the pose belongs to, if any
device_id	text	Pseudonymous device label, 1 to 120 characters; "fake-test" for pipeline tests
provider_map_id	text, nullable	VPS map the phone localized against; must match `vps_map_id` when both are set
provider_pose	provider_pose	Raw pose from the phone (1.1)
world_position	position, nullable	Converted by the server: (x, y, −z)
heading_deg	number, nullable	0 to 360; 0 = world +Z, increasing toward world +X
tracking_status	enum	`localizing`, `valid`, `lost`
data_origin	enum	`real`, `synthetic`; decided by the server: synthetic for the fake-test device or when the client sends synthetic, otherwise real
captured_at	timestamp	Time on the phone; at most 1 minute in the future
received_at	timestamp	Time on the server
session_id, transform_id, world_orientation	UUID, UUID, orientation	Planned for M3

A pose is **live** when its tracking is `valid` and it is less than 5 seconds old; otherwise /live shows it as “Not live”. Synthetic poses are labeled “test pose” on /live but, in M2, still drive the live dot and route. From M3, stale, lost and synthetic poses cannot start a new live route.

navigation_sessions [Planned]

Field	Type	Notes
session_id	UUID	Primary key
user_id	UUID, nullable	Null only for seeded test sessions
site_id, map_version_id	UUID	Map is fixed for the whole session

Field	Type	Notes
device_id	text	Pseudonymous device label
device_model	text	e.g. "iPhone 17 Pro"; "synthetic" for test data
sdk_version	text, nullable	Niantic SDK version used
mode	enum	live, simulation, replay
data_origin	enum	live_sdk, synthetic, recorded
status	enum	active, ended, failed
started_at	timestamp	When the session started
ended_at	timestamp, nullable	Null while active

1.11 Routes [M2 partial] and simulated agent [Planned]

In M2, POST /api/route computes a route from a start node to a destination, for a person or the simulated robot, and returns it without storing it. **From M3**, routes are stored in routes so they can be invalidated and replanned.

Field	Type	Notes
route_id	UUID	M3+ (stored routes only)
session_id	UUID, nullable	M3+
map_version_id	UUID	Map the route was computed on
graph_revision, path_state_revision	integer	Graph and open/closed state used
start_mode	enum	manual in M2; live from M3
start_node_id	UUID	M2: chosen by hand on /viewer, or the node nearest the phone on /live
start_position	position, nullable	M3+: live position used as the start
start_segment_id	UUID, nullable	M3+: walkable segment the live start is attached to
destination_id	UUID	Destination requested; resolved to its node by the API
profile	enum	human, robot
steps	array of route_step	Ordered path; empty when unreachable or when start = destination
total_distance_m	number, nullable	0 when start = destination; null if unreachable
status	enum	valid, unreachable (M2); invalidated, arrived (M3+)
created_at	timestamp	When the route was computed (server time)

simulated_agents [Planned]

Field	Type	Notes
agent_id	UUID	Primary key
session_id	UUID	Simulation-mode session only
route_id	UUID, nullable	Current route
position	position	Current position on its path
current_step_index	integer, nullable	Index into the route's steps
step_progress_m	number	Distance along the current step
speed_mps	number	Configured simulation speed (> 0)

Field	Type	Notes
state	enum	idle, running, paused, waiting_for_operator, arrived, unreachable
updated_at	timestamp	When the position or state last changed

A live start (M3) is accepted only if it is within max_start_distance_m (a site setting, agreed after first tests) of a walkable segment on the same map version and the pose is live; otherwise it is rejected with an error. The robot is always labeled SIMULATED in the UI. When a segment's availability or robot access changes, affected routes are invalidated and replanned, or marked unreachable.

1.12 Visual assets [M2 partial] and alignment [Planned]

In M2, the HSS 349 model is a static GLB file served by Vercel (public/models/hss349-v2.glb, 4.9 MB, textures 2048 px WebP, reduced from the 14.5 MB export). No frame transform is needed because the model, the graph and the poses share the VPS frame (1.1). Both tables below are created in M3, when models move to Supabase Storage and more maps are added.

visual_assets [Planned]

Field	Type	Notes
asset_id	UUID	Primary key
map_version_id	UUID	Map version the asset displays
asset_kind	enum	mesh, preview_image
storage_path	text	Supabase Storage key
file_format	enum	glb, png, jpeg, webp
byte_size	integer	Must be within the limits below
checksum_sha256	text	64 hex characters
license_reference	text	Who captured it / permission source
created_at	timestamp	When the asset was uploaded

frame_transforms [Planned]

Field	Type	Notes
transform_id	UUID	Primary key
map_version_id	UUID	Map version the transform belongs to
source_frame_id	text	e.g. a VPS site frame
target_frame_id	text	The map's world frame
matrix_4x4	array of 16 numbers	Row-major source → world transform; must be invertible
validation_status	enum	unverified, verified, rejected
verified_at	timestamp, nullable	When the transform was checked; required when verified
verification_notes	text	Which known points were checked and the error found

Media formats and limits

Asset	Format	Max size
3D mesh	GLB (textures JPEG or WebP, max 4096 px)	100 MB per file; keep under 10 MB for phones
Images	PNG, JPEG, WebP	10 MB
Video	Not stored	n/a

Total spatial asset storage stays within the 1 GB budget from M1. Raw camera recordings are not stored. From M3, live navigation uses only `verified` transforms.

1.13 Path events, simulated obstacles and agent reports [Planned]

path_events

Field	Type	Notes
`event_id`	UUID	Primary key
`map_version_id`, `segment_id`	UUID	Map version and segment that changed
`actor_user_id`	UUID	Operator or admin who made the change
`event_type`	enum	`availability_change`, `robot_access_change`
`old_value`, `new_value`	text	Must be valid values of that field and must differ
`reason`	text	Why the change was made, e.g. "wet floor"; required
`path_state_revision`	integer	New map-wide revision number
`source_report_id`	UUID, nullable	Agent report that caused it, if any
`committed_at`	timestamp	When the change was committed

agent_reports

Field	Type	Notes
`report_id`	UUID	Primary key
`agent_id`, `segment_id`	UUID	Robot that reported it and the blocked segment
`reason`	text	Tester-injected simulated obstacle, not real sensing
`status`	enum	`pending`, `confirmed`, `rejected`
`reported_at`	timestamp	When the robot sent the report
`reviewed_by_user_id`	UUID, nullable	Required once reviewed
`reviewed_at`	timestamp, nullable	When the operator decided; required once reviewed
`closure_event_id`	UUID, nullable	Required when confirmed

simulated_obstacles

Field	Type	Notes
`obstacle_id`	UUID	Primary key
`session_id`	UUID	Simulation session
`segment_id`	UUID	Segment the tester blocked
`placed_by_user_id`	UUID	Tester who placed it
`is_active`	boolean	Set false when dismissed
`placed_at`	timestamp	When the tester placed it
`dismissed_at`	timestamp, nullable	When it was dismissed; null while active

When the robot reaches the node before a segment with an active obstacle, it stops (`waiting_for_operator`) and creates an agent report. **Confirm:** the segment closes and routes replan. **Reject:** the obstacle is dismissed (`is_active = false`) and the robot resumes if its route is still valid. A pending report never closes a path. Confirmation creates exactly one closure event; confirming twice creates no duplicate. Stale writes are rejected.

1.14 Metric snapshot and localization health [Planned]

metric_snapshots

Field	Type	Notes
`snapshot_id`	UUID	Primary key
`test_point_id`, `map_version_id`	UUID	Health is per point, device and map
`device_id`	text	Device the health applies to
`attempt_ids`	array of UUID	Exact attempts used
`attempt_count`	integer	All completed live attempts, including failures; in_progress excluded
`success_count`	integer	Attempts with outcome success
`success_rate`	number, nullable	0–1; null if no attempts
`median_latency_ms`	number, nullable	Successful attempts only
`valid_error_count`	integer	Attempts with a valid reference error
`median_horizontal_error_m`	number, nullable	Null if no valid errors
`health_status`	enum	`gray`, `red`, `yellow`, `green`
`calculated_at`	timestamp	When the snapshot was computed

Health uses only **completed live** attempts (in_progress is excluded) from the same test point, device and map version. The rules are applied in order and the first match wins:

Order	Status	Rule
1	Gray	Fewer than 10 attempts
2	Red	≥10 attempts and success below 60%
3	Red	≥10 attempts and ≥5 valid error samples and median error above 1.5 m
4	Gray	≥10 attempts but fewer than 5 valid error samples
5	Green	≥10 attempts, ≥5 valid error samples, success ≥80% and median error ≤1 m
6	Yellow	All other cases

Counts are always shown with the status. Health describes tested positions only, not map freshness.

1.15 AI summaries and exports [Planned]

ai_summaries

Field	Type	Notes
`summary_id`	UUID	Primary key
`snapshot_id`	UUID	Metrics the summary explains
`requested_by_user_id`	UUID	User who asked for the summary
`model_version`, `prompt_version`	text	Gemini model and prompt template used, for reproducibility
`status`	enum	`pending`, `completed`, `failed`, `timed_out`
`observations`	array of {text, metric_fields, attempt_ids}	Each claim cites its source
`suggestions`	array of text	Labeled separately from observations

Field	Type	Notes
`requested_at`	timestamp	When the summary was requested
`completed_at`	timestamp, nullable	When Gemini answered or the request failed; null while pending

attempt_exports

Field	Type	Notes
`export_id`	UUID	Primary key
`requested_by_user_id`	UUID	User who requested the export
`filters`	object	Test point, outcome, mode and date range used
`file_format`	enum	`csv`, `json`
`row_count`	integer	Number of attempts in the file
`generated_at`	timestamp	When the file was generated

Gemini receives only aggregate metrics and attempt IDs, never camera data. All numbers are computed by the server; generated numbers that do not match the snapshot are rejected. The dashboard keeps working if Gemini fails (15-second timeout).

1.16 Relationships

- A **site** has map versions; a **map version** owns its graph nodes, path segments, destinations, test points, pose samples, assets and transforms.
- A **path segment** connects two graph nodes in the same graph revision; a **destination** points to one node.
- A **localization attempt** belongs to one test point and map version (and to a session from M3).
- A **pose sample** belongs to one map version and optionally to one attempt (and to a session from M3).
- A **route** uses ordered path segments from one graph revision and one path-state revision.
- An **agent report** targets one path segment; confirmation links it to one **path event**.
- A **metric snapshot** records the exact attempts it was computed from; an **AI summary** cites one snapshot.
- History is preserved: attempts, events and referenced map versions are retired, not deleted.

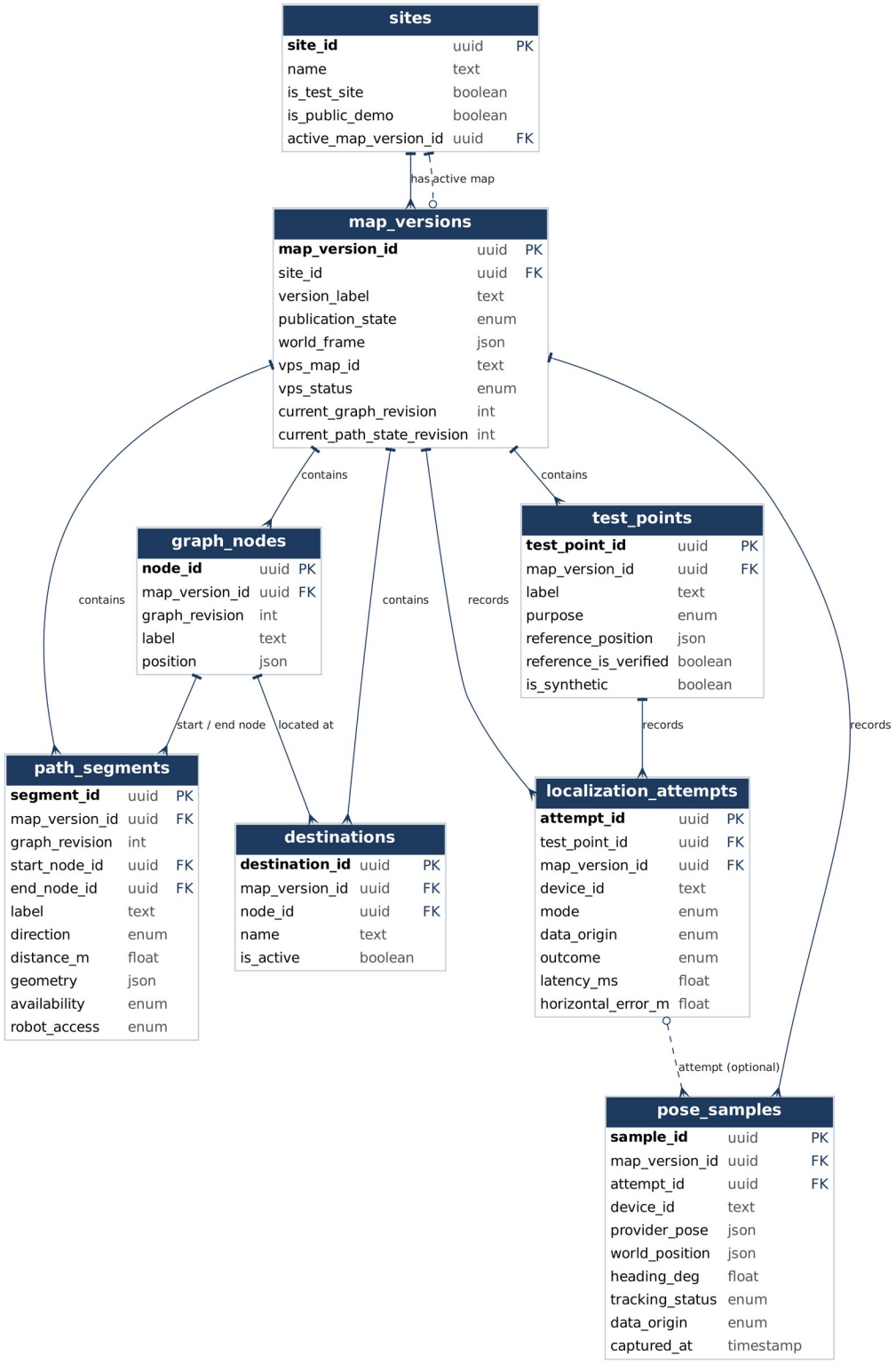


Figure 1.1: M2 database tables (key fields)

1.17 Traceability to requirements

Data concept	Section	Requirements (Section 2)
Users, roles and privileges	1.2	R1.1–R1.4, R7.1–R7.3, R8.1–R8.2
Site and map version	1.3–1.4	R2.1, R2.2
Navigation graph	1.5–1.6	R2.3, R5.2, R5.4, R7.1, R7.2, R8.1–R8.3, R15.3, R17.1, R17.2
Destination	1.7	R2.4, R5.1, R7.3, R15.3
Test point and reference	1.8	R12.1, R14.3
Localization attempt	1.9	R3.4, R12.1, R12.2, R14.1–R14.4, R15.1
Pose sample and session	1.10	R3.2, R3.3, R3.5, R4.1, R4.2, R6.2–R6.5, R11.1, R11.2
Route and simulated agent	1.11	R5.1–R5.5, R6.1, R9.1–R9.3, R10.1–R10.5
Visual asset and transform	1.12	R2.2, R3.3
Path event, obstacle and report	1.13	R8.1–R8.3, R17.2, R18.1–R18.6
Metric snapshot and health	1.14	R13.1, R13.2, R14.1–R14.5
AI summary and export	1.15	R15.2, R16.1

2. Functional Requirements V2

Parent IDs R1–R18 are kept from M1, and the sub-IDs match the list sent to the professor on Oct 8. New items R3.5, R6.5 and R15.3 cover built features that had no ID, and R6.2 is split into R6.2.1 and R6.2.2.

- **Priority:** 1 = must have, 2 = desired, 3 = opportunistic. Priority 1 items are our commitment.
- **Target:** the milestone we planned on Oct 8. Every priority 1 item is targeted by M4, leaving M5 for integration, testing and Demo Day.
- **Status (Oct 20):** Done (M2) = planned for M2 and working on the production site; Done early = planned for a later milestone but already built and deployed (the note says how it was verified); New – done = added and working; Partial = part of it works (the note says which part); Planned = not built yet.
- **Pages:** /viewer is the 3D planning view (scan, graph, manual start, person or robot route). /live is the 2D live navigation view (live dot, automatic start, arrival). The home page is the M2 list/search test page.
- **Terms:** "path segment" is the walkable link between two nodes ("edge" in M1), as in Section 1 and the path_segments table.

2.1 Requirements

ID	Requirement	Pri.	Target	Status (Oct 20) and where
R1	**Access control**			
R1.1	User can sign up	1	M3	Planned
R1.2	User can log in	1	M3	Planned
R1.3	Each user has one active role per site: visitor, developer, operator or administrator	1	M3	Planned
R1.4	Only operators/admins can change path segments and destinations	1	M3	Planned – M2: data is edited only by team members in Supabase
R2	**Shared environment**			
R2.1	Capture one SFSU indoor area with Scaniverse, process it, and activate it as a VPS location	1	M2	Done (M2) – Site "SFSU HSS 349", map v2 is Production
R2.2	Show the Scaniverse 3D model of the area in the web app	1	M2	Done (M2) – /viewer (GLB hosted on Vercel)
R2.3	Show navigation graph nodes and path segments on the 3D model	1	M2	Done (M2) – /viewer
R2.4	Show destinations as labeled markers (HSS 349: 4, one per node)	1	M2	Done (M2) – /viewer
R3	**Live localization**			
R3.1	Phone app localizes itself in the site using Niantic VPS	1	M2	Done (M2) – iPhone; PRECISE + TRACKING in HSS 349
R3.2	Phone sends its pose (position, rotation, tracking state, timestamp) to the server	1	M3	Done early – POST /api/poses → pose_samples, device token, up to 2 per second. Verified on production Oct 20: 828 pose samples across 8 upload periods (evidence/live-position.md)
R3.3	Show the person's live position and heading as a dot on the web map, updating while they walk	1	M3	Done early – /live, via Supabase Realtime. Verified on production Oct 20, on a second phone and several browsers
R3.4	Stop a localization attempt after 30 seconds with no result and record it as a timeout	1	M3	Planned
R3.5	Label test (synthetic) poses on the live map as "test pose, not a real localization"	2	M2	New – done – /live; uses pose_samples.data_origin

ID	Requirement	Pri.	Target	Status (Oct 20) and where
R4	**Position status**			
R4.1	Show tracking state: localizing, valid or lost	1	M3	Partial – Live / Not live display works; the phone reports only valid in M2. Full tracking states from the phone in M3
R4.2	Mark the position as stale (“Not live”) if it is older than 5 seconds	1	M3	Done early – /live
R5	**Routing**			
R5.1	User selects a destination	1	M2	Done (M2) – /viewer, /live
R5.2	Compute the shortest route over open path segments (human)	1	M2	Done (M2) – Dijkstra; POST /api/route
R5.3	Draw the route on the scanned 3D model; also show it on the 2D live map	1	M2	Done (M2) – /viewer (3D), /live (2D)
R5.4	Robot route uses only open, robot-allowed path segments	1	M3	Done early – /viewer route only; robot movement is R10
R5.5	Show route distance in meters	2	M4	Done early – /viewer, /live
R6	**Route start**			
R6.1	User can pick a start node manually	1	M2	Done (M2) – /viewer, /live
R6.2	Use the live position as the route start:			
R6.2.1	Start at the node nearest the live position; switch only when another node is at least 0.75 m closer	1	M3	Done early – /live “My current location (auto)”
R6.2.2	Attach the live position to the nearest path segment on the same map, within a set distance	1	M3	Planned
R6.3	Reject a start that is too far from any path segment or on a different map, with an error message	1	M3	Planned
R6.4	Block a new live route when the position is stale or lost	1	M3	Planned
R6.5	Show “You have arrived” when a live, valid position is within 1.0 m of the destination	2	M2	New – done – /live
R7	**Map administration**			
R7.1	Operator adds a node by clicking on the 3D model	2	M4	Planned
R7.2	Operator connects two nodes with a path segment	2	M4	Planned
R7.3	Operator renames a destination	3	M5	Planned – If time
R8	**Shared closures**			
R8.1	Operator closes a path segment	1	M3	Planned – Routing already skips closed segments
R8.2	Operator reopens a path segment	1	M3	Planned
R8.3	Path segment open/closed changes appear on every open viewer in real time	1	M3	Planned
R9	**Route revision**			
R9.1	Recalculate active routes after a path segment closes	1	M3	Planned

ID	Requirement	Pri.	Target	Status (Oct 20) and where
R9.2	Recalculate an active robot route after a path segment on it becomes robot-restricted	1	M3	Planned
R9.3	Show “Unreachable” when no permitted route exists	1	M3	Partial – Shown for a new route request; active routes in M3
R10	**Robot simulation**			
R10.1	Start the simulated robot on a route	1	M3	Planned
R10.2	Show the robot moving along its route, always labeled SIMULATED	1	M3	Planned
R10.3	Robot replans after a path segment on its route closes	1	M3	Planned
R10.4	Pause the robot	2	M4	Planned
R10.5	Reset the robot to its start	2	M4	Planned
R11	**Replay**			
R11.1	Record a walk’s poses	3	M5	Partial – Every pose is stored in pose_samples; no start/stop recording yet
R11.2	Replay a recorded walk on the map	3	M5	Planned – If time
R12	**Attempt logging**			
R12.1	Save every localization attempt with its test point and outcome	1	M3	Planned – M2 uses labeled synthetic attempts
R12.2	Save failures and timeouts with a reason	1	M3	Planned
R13	**Evaluation dashboard**			
R13.1	Show a success-rate chart per test point	2	M4	Planned
R13.2	Show a horizontal-error chart per test point	2	M4	Planned
R14	**Localization health**			
R14.1	Count localization attempts per test point	1	M4	Planned
R14.2	Compute success rate per test point	1	M4	Planned
R14.3	Compute median horizontal error per test point, using only valid independent reference measurements	1	M4	Planned
R14.4	Count valid error samples per test point	1	M4	Planned
R14.5	Color each test point green, yellow, red or gray using the M1 rules (uses R14.1–R14.4)	1	M4	Planned
R15	**Search and export**			
R15.1	List and search localization attempts by test point, outcome and text (M2 uses labeled synthetic records)	1	M2	Done (M2) – Home page; GET /api/localization-attempts
R15.2	Export attempts as CSV	3	M5	Planned – If time
R15.3	List destinations and path segments (status, robot access) for each public map	1	M2	New – done – Home page; GET /api/graph
R16	**AI summary**			
R16.1	Gemini writes a plain-English summary of the health results	2	M4	Planned

ID	Requirement	Pri.	Target	Status (Oct 20) and where
R17	**Robot access**			
R17.1	Operator marks a path segment as robot-restricted	1	M3	Planned – robot_access already stored and used by routing
R17.2	Restriction is kept when the path segment is closed and reopened	1	M3	Planned
R18	**Robot blockage**			
R18.1	Tester places a simulated obstacle on a path segment during a run	2	M4	Planned
R18.2	Robot stops at the node before the blocked path segment	2	M4	Planned
R18.3	Robot sends a blockage report	2	M4	Planned
R18.4	Operator gets a real-time alert	2	M4	Planned
R18.5	Operator confirms: the path segment closes (then R8.3 and R9.1 apply)	2	M4	Planned
R18.6	Operator rejects: the obstacle is dismissed and the robot resumes if its route is still valid	3	M5	Planned – If time

2.2 Summary

Priority	Done	Partial	Planned	Total
1 – must have	16	2	25	43
2 – desired	3	0	12	15
3 – opportunistic	0	1	4	5
Total	**19**	**3**	**41**	**63**

19 of 63 requirements already work, 3 work in part, and 41 are planned. 25 priority 1 items are still planned, all targeted at M3 or M4.

2.3 Changes since the Oct 8 list

- Priorities are written as 1/2/3 instead of High/Mid/Low, and a Status column was added.
- “Edge” is now “path segment” everywhere, to match Section 1 and the database.
- Built earlier than planned: R3.2 pose upload, R3.3 live dot, R4.2 stale position, R5.4 robot route, R5.5 route distance and R6.2.1 automatic start. The professor asked on Oct 13 to see live position change along a route, so live position and routing are now connected on /live in M2 instead of M3. R4.1 is partial: /live shows Live or Not live, but the phone does not yet report localizing or lost.
- R2.4: HSS 349 has 4 destinations, one per graph node. The synthetic demo map keeps its 3 destinations.
- R6.2 split: R6.2.1 uses the nearest node (built). R6.2.2 attaches to the nearest path segment (M3).
- New: R3.5 test-pose label, R6.5 arrival message, R15.3 list of destinations and path segments.
- R15.1 raised from 2 to 1, because the list/search prototype is a required part of M2.
- The phone app runs on iPhone (Unity 6 + Niantic Spatial SDK 4.1), not Android as stated in M1.

2.4 What the M2 demo shows

- **Scan and VPS:** HSS 349 captured with Scaniverse and active as a VPS location; the iPhone localizes in it (R2.1, R3.1).

- **3D planning (/viewer):** the scan with nodes, path segments and 4 labeled destinations; pick a start and destination; person or robot route drawn with its distance (R2.2–R2.4, R5.1–R5.5, R6.1).
- **Live navigation (/live):** the phone's position and heading move on the map, the route starts from the nearest node, and "You have arrived" appears at the destination (R3.2, R3.3, R4.2, R6.2.1, R6.5; R4.1 in part).
- **List/search (home page):** destinations and path segments for each map, and search of localization attempts by test point, outcome and text (R15.1, R15.3).

2.5 Plan to M5

The plan follows the target milestones in 2.1 and the course milestones: M3 = prototype review and final function commitment, M4 = QA and usability testing, M5 = final delivery and Demo Day. Still to do: 22 items targeted at M3 (20 planned, 2 partial), 17 at M4, and 5 at M5 (if time). The first name in Leads leads the item; leads will be confirmed when the M3 commitment is set.

Milestone	Goal	Requirements	Leads
M3	Accounts and roles	R1.1–R1.4	Noah
M3	Live position rules: timeout, tracking states, start on nearest path segment, reject or block bad starts	R3.4, R4.1, R6.2.2, R6.3, R6.4	Sumit
M3	Shared closures, rerouting and robot access	R8.1–R8.3, R9.1–R9.3, R17.1–R17.2	Noah, Sumit
M3	Robot simulation: start, move with SIMULATED label, replan	R10.1–R10.3	Ezeren, Sumit
M3	Real localization attempt logging	R12.1–R12.2	Noah, Tyler
M3	Fixes from Section 6: limit pose reads, measure live delay, phone logging, pose API next to the database, rotate device token	–	Sumit, Ezeren
M4	Localization health, charts and dashboard	R13.1–R13.2, R14.1–R14.5	Calvin, Sumit
M4	AI summary of health results	R16.1	Sumit
M4	Operator map editing	R7.1–R7.2	Noah, Ezeren
M4	Robot pause/reset and blockage reports	R10.4–R10.5, R18.1–R18.5	Noah, Ezeren
M4	Test-point trials in HSS 349, QA plan and usability testing; scan the Demo Day room if it is confirmed to be elsewhere	–	Tyler, Calvin
M3–M5	Code review of every pull request to dev, technical guidance, setting the M3 commitment	–	Daksh
M5	Optional items if time: rename destination, replay, CSV export, reject a blockage	R7.3, R11.1–R11.2, R15.2, R18.6	Owners above
M5	Integration and regression testing, recorded backup demo, revised M1–M4 documents, Demo Day	–	Whole team, Daksh leads

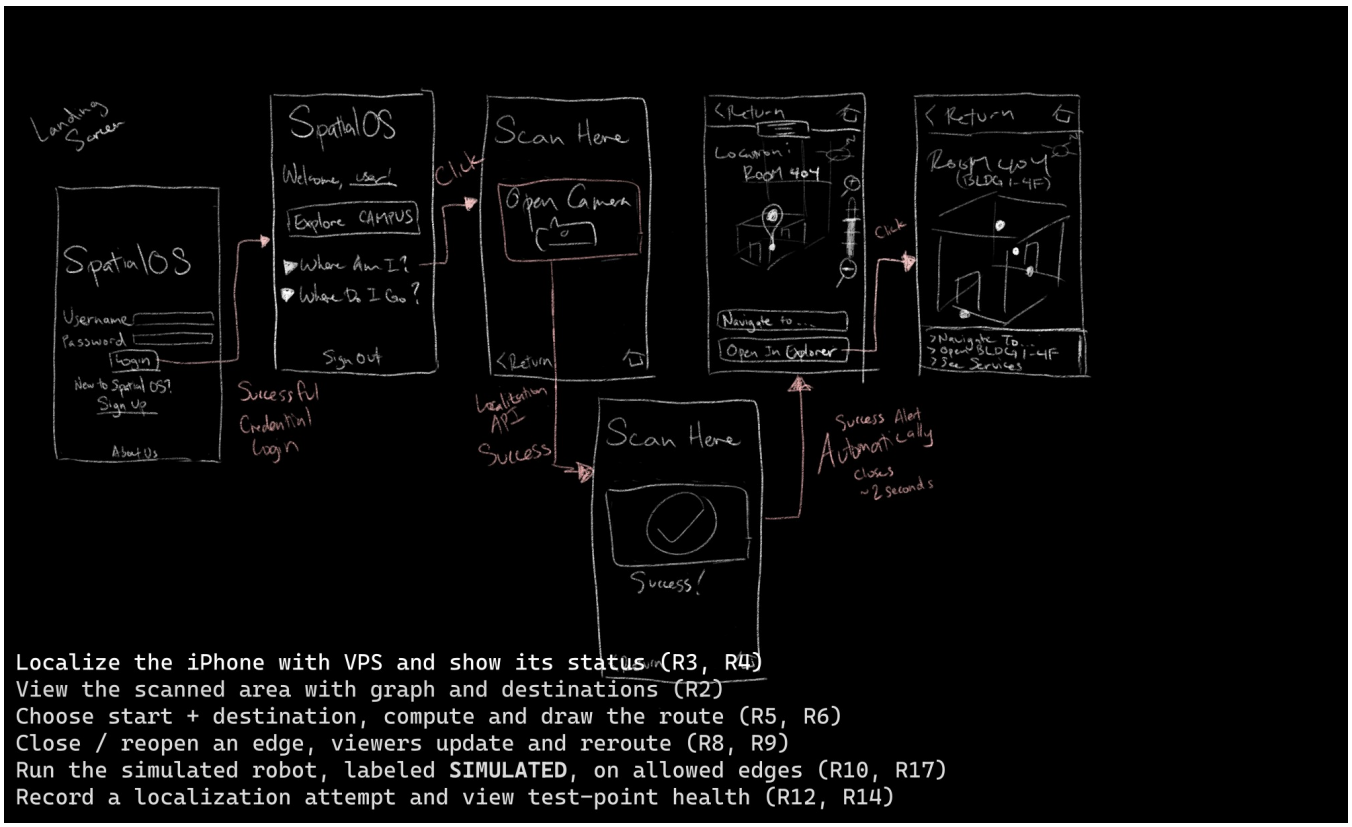
3. UI Mockups and UX Flows

3.1 Major functional requirements: UX flows

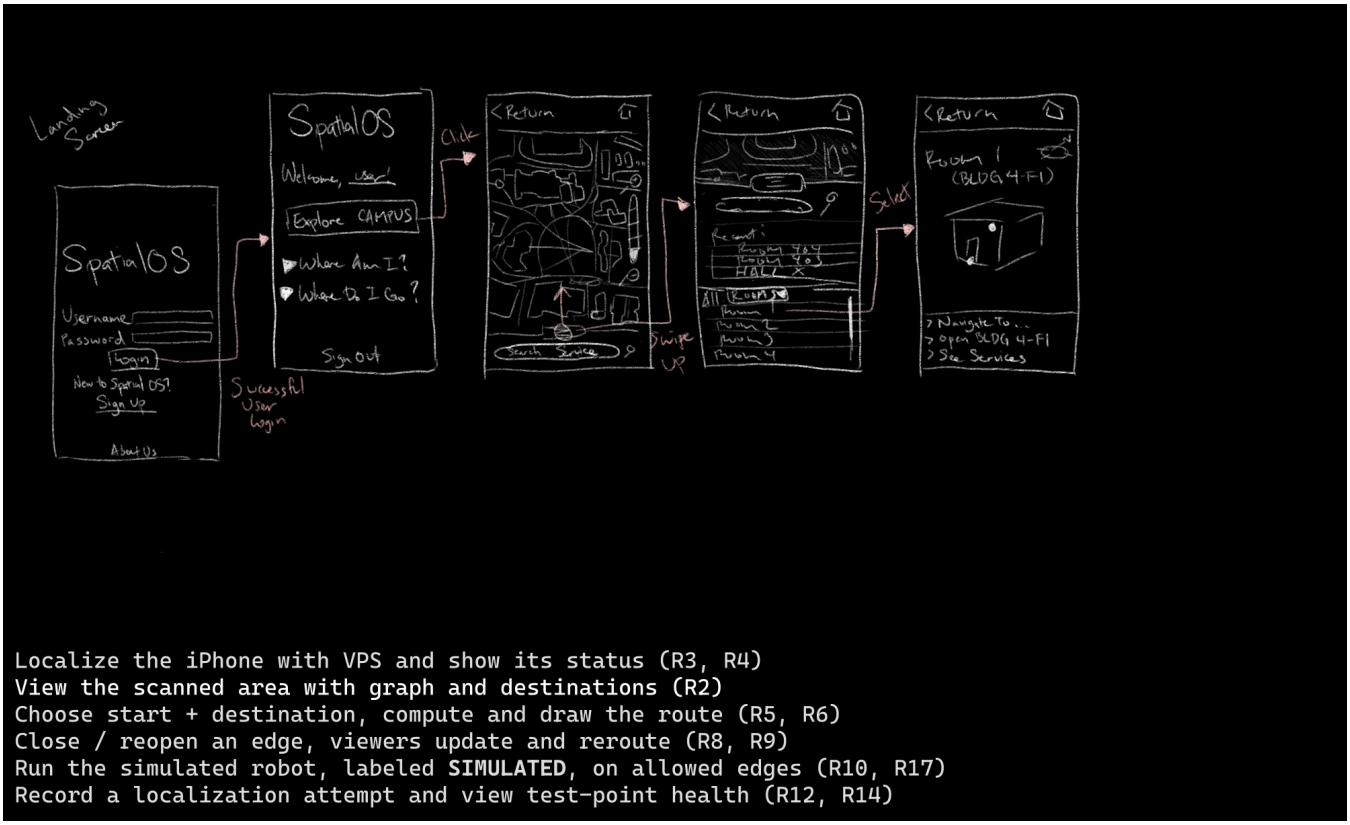
The six flows below are hand-drawn wireframes of the main user journeys. Each flow is read left to right; arrows show the action that moves the user to the next screen. Wide flows are shown in two parts. Flows 2, 4, 5 and 6 are followed by detailed screens with numbered steps, which show each state change; those screens also mark which steps are built in M2 and which are planned.

- Localize the iPhone with VPS and show its status (R3, R4)
- View the scanned area with graph and destinations (R2)
- Choose start + destination, compute and draw the route (R5, R6)
- Close / reopen a path segment, viewers update and reroute (R8, R9)
- Run the simulated robot, labeled SIMULATED, on allowed path segments (R10, R17)
- Record a localization attempt and view test-point health (R12, R14)

Flow 1: Localize the iPhone with VPS and show its status (R3, R4)



Flow 2: View the scanned area with graph and destinations (R2)

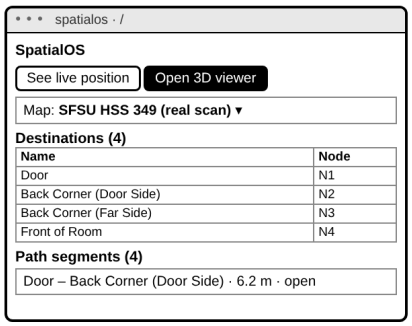


Localize the iPhone with VPS and show its status (R3, R4)
View the scanned area with graph and destinations (R2)
Choose start + destination, compute and draw the route (R5, R6)
Close / reopen an edge, viewers update and reroute (R8, R9)
Run the simulated robot, labeled **SIMULATED**, on allowed edges (R10, R17)
Record a localization attempt and view test-point health (R12, R14)

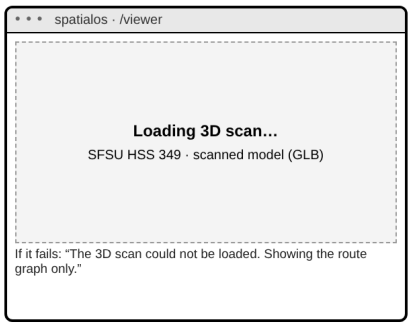
Detailed screens, Flow 2

Flow 2 — View the scanned area with graph and destinations (R2.2–R2.4, R15.3)

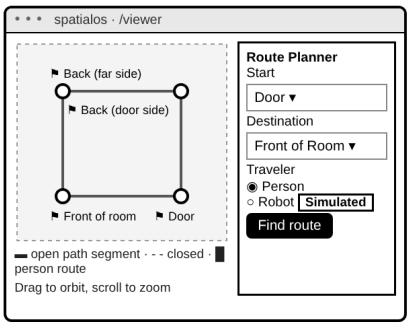
Detailed screens of the current web app (graph drawn as a schematic, not to scale). Example data: the real HSS 349 map (4 nodes, 4 path segments, 4 destinations).



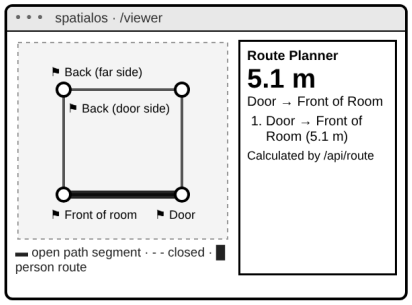
1 Home page: choose a public map; its destinations and path segments are listed from the database (R15.3). "Open 3D viewer" opens the scan.



2 The viewer loads the scanned model; if it fails, the graph and routes are still shown.



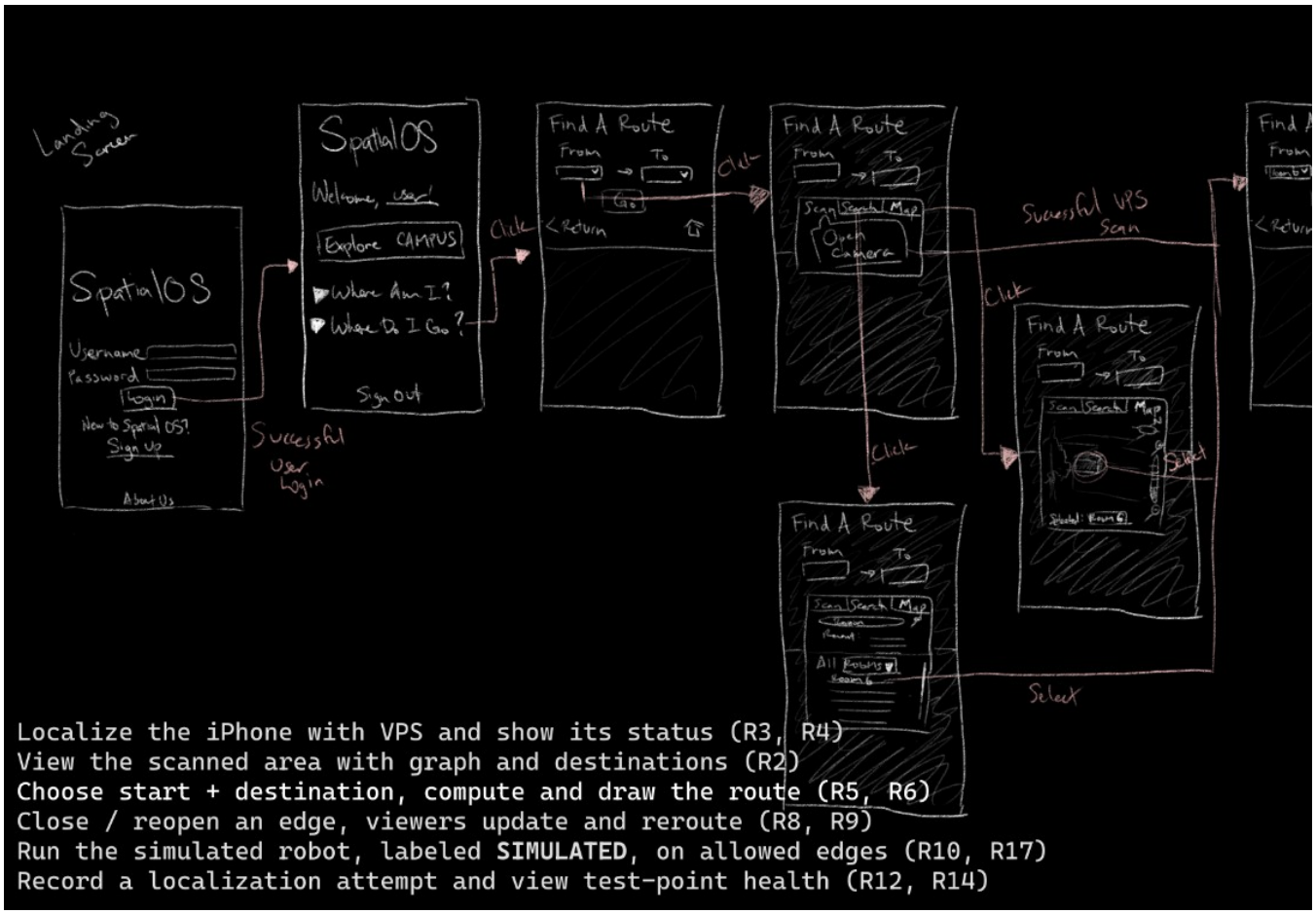
3 Scan with graph nodes, path segments and labeled destinations (R2.2–R2.4). Start and destination are chosen in the Route Planner menu.



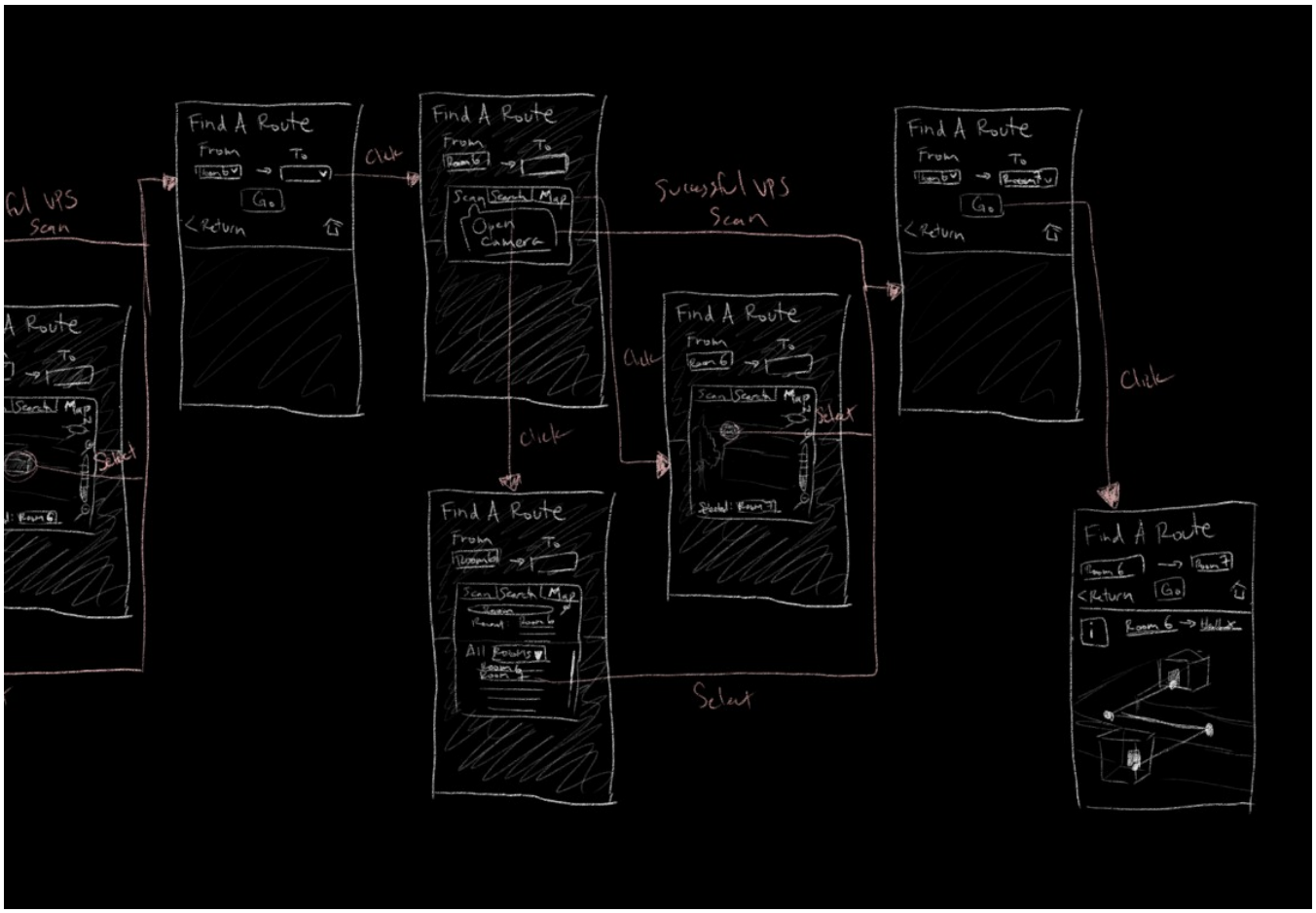
4 "Find route" draws the route on the scan with its distance and steps (R5.1–R5.3, R5.5; continues in Flow 3).

Status: Built in M2. These screens match the current home page and /viewer (Route Planner with dropdowns, Person/Robot traveler, Find route). Clicking markers on the scan and layer toggles are not built; they are proposed for later milestones.

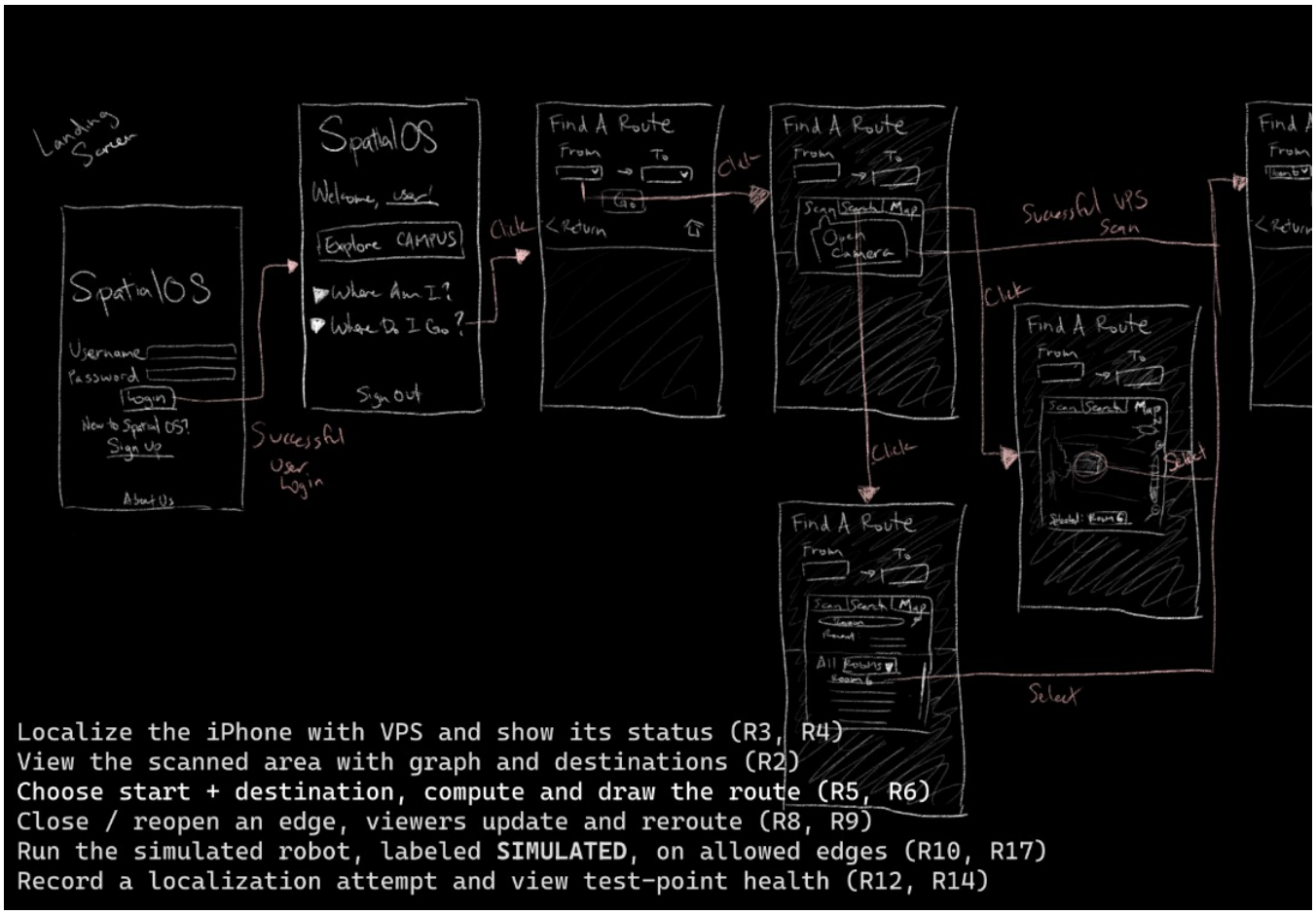
Flow 3: Choose start + destination, compute and draw the route (R5, R6)



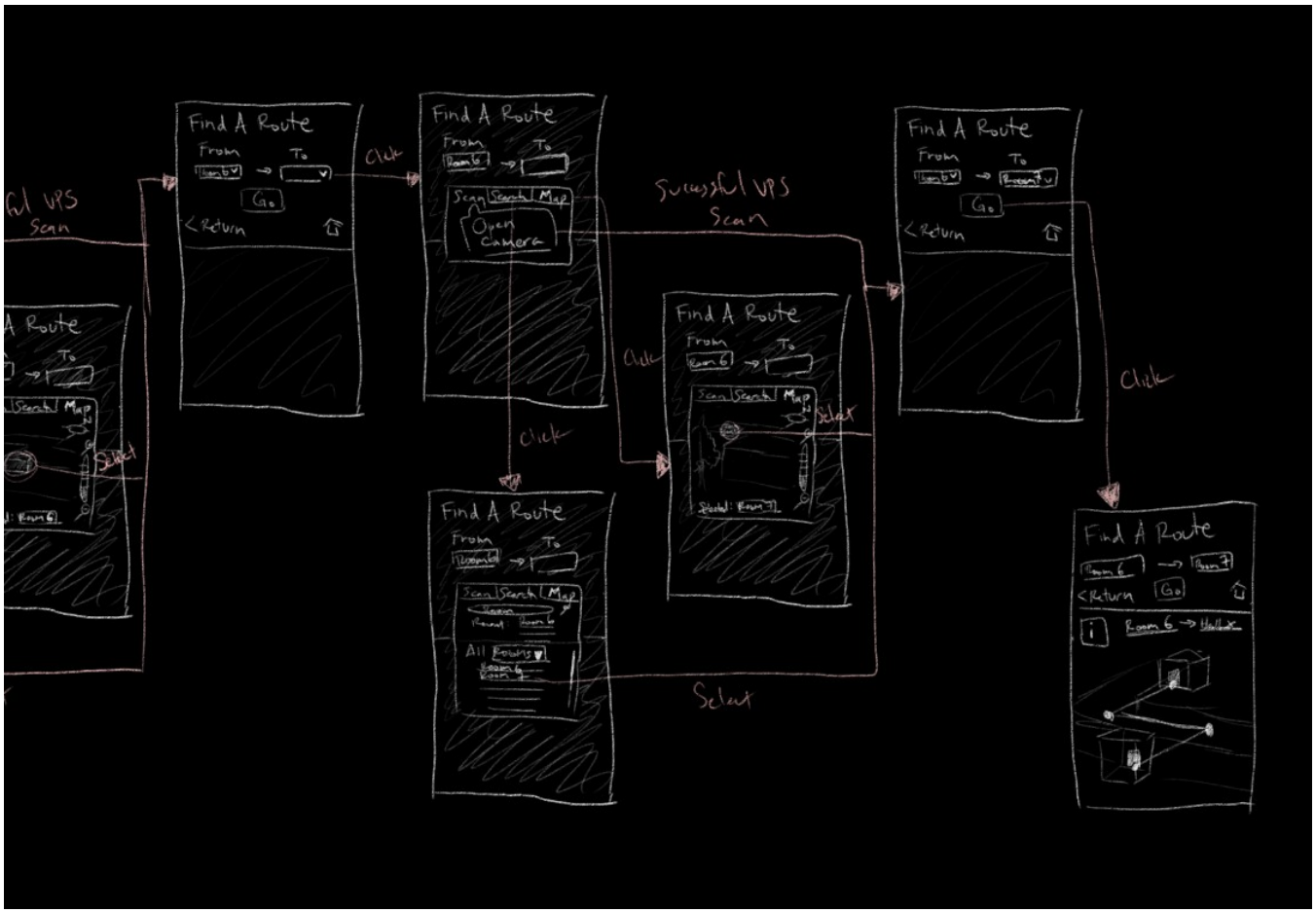
Flow 3 (continued)



Flow 4: Close / reopen a path segment, viewers update and reroute (R8, R9)



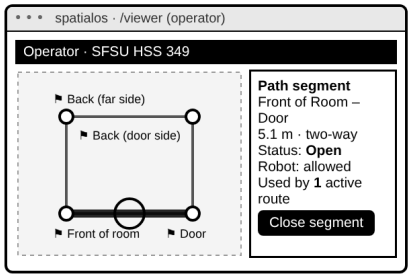
Flow 4 (continued)



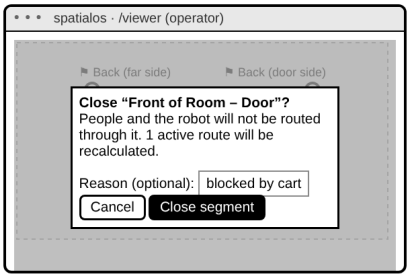
Detailed screens, Flow 4

Flow 4 — Close / reopen a path segment, viewers update and reroute (R8.1–R8.3, R9.1, R9.3)

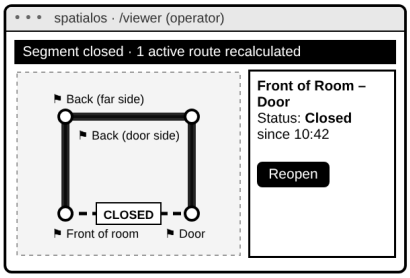
Detailed screens (graph drawn as a schematic of HSS 349, not to scale). An operator closes the Front of Room – Door segment while a visitor is navigating Door → Front of Room.



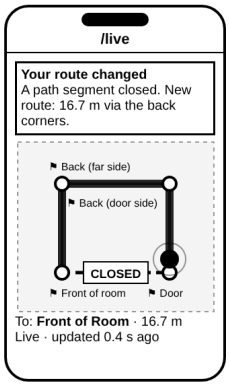
1 Operator selects a path segment. The panel shows its status and how many active routes use it.



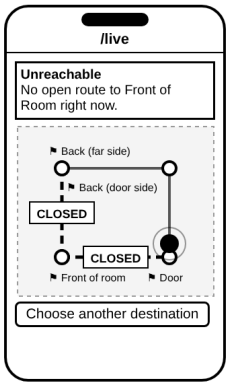
2 Confirmation explains the effect before anything changes.



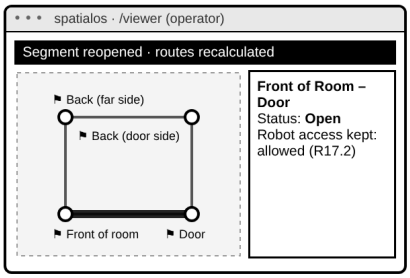
3 Segment is drawn dashed and labeled CLOSED; affected routes are recalculated (R8.1, R9.1).



4 Every open viewer updates in real time; the visitor sees why the route changed (R8.3).



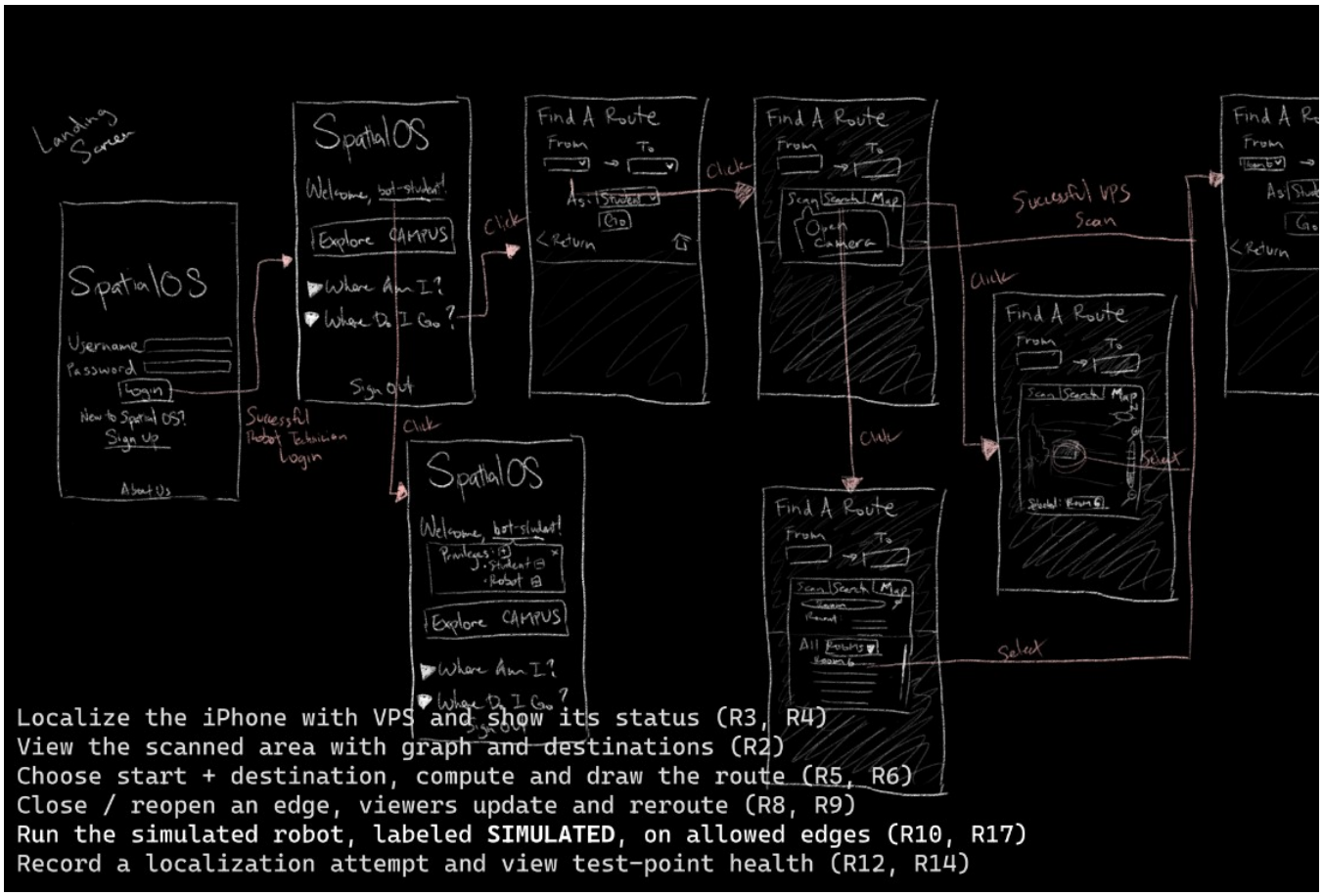
5 If no permitted route remains, the app shows “Unreachable” instead of a broken route (R9.3).



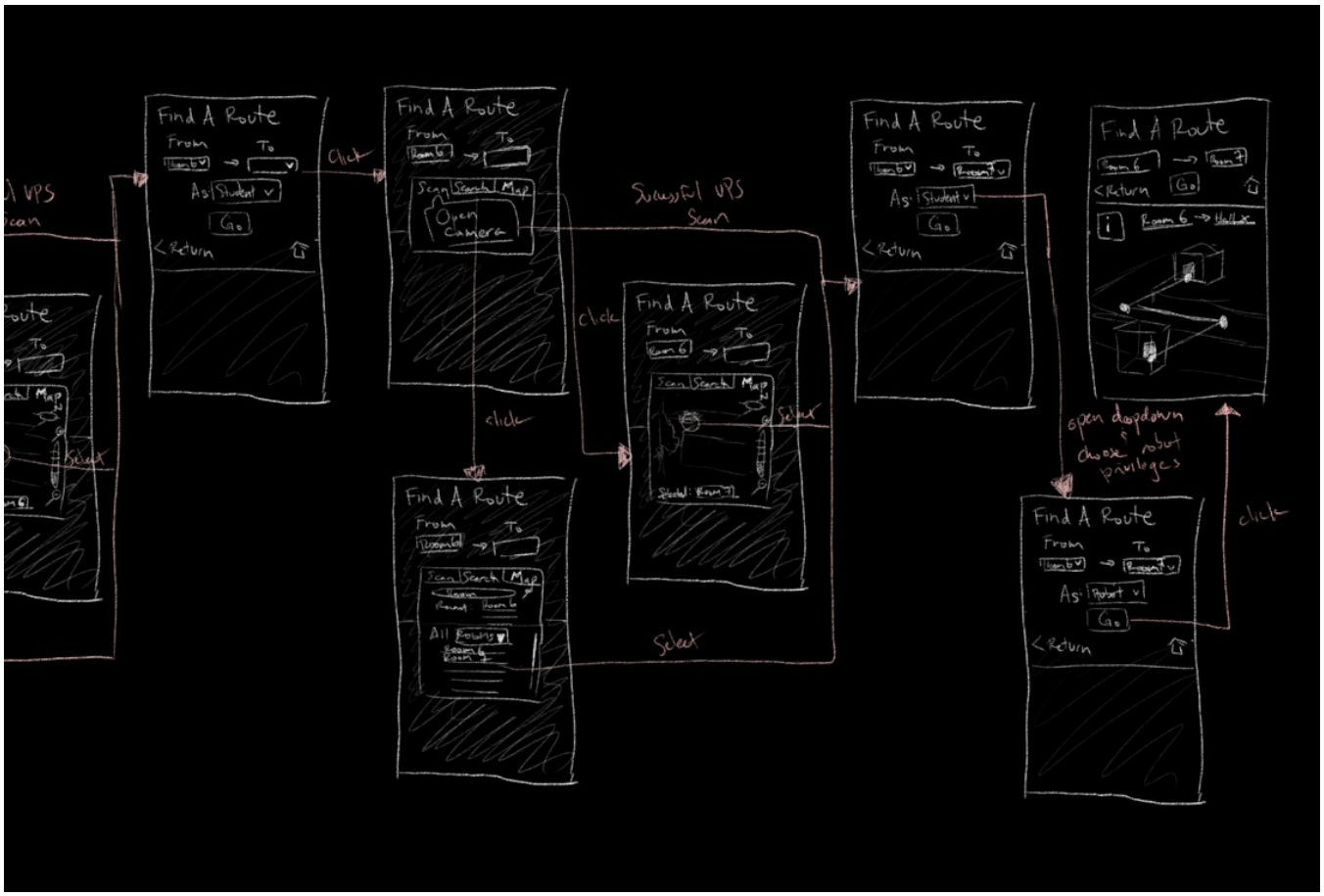
6 Operator reopens the segment; routes are recalculated again and robot access is unchanged (R8.2, R17.2).

Status: Planned for M3 (R8.1–R8.3, R9.1). Already in M2: routing skips closed segments, and “Unreachable” is shown for a new route request (R9.3 partial).

Flow 5: Run the simulated robot, labeled SIMULATED, on allowed path segments (R10, R17)



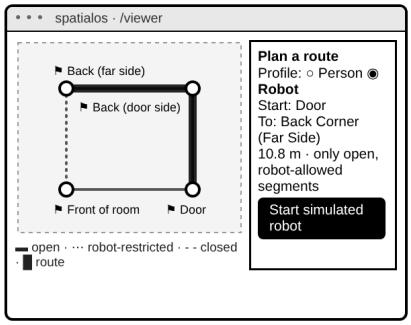
Flow 5 (continued)



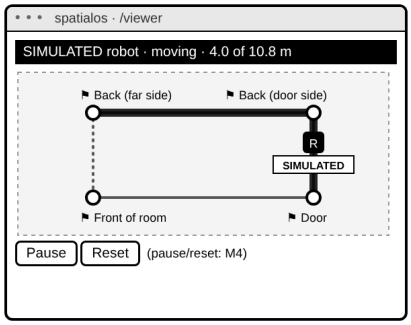
Detailed screens, Flow 5

Flow 5 — Run the simulated robot, labeled SIMULATED, on allowed path segments (R10.1–R10.3, R5.4, R17.1)

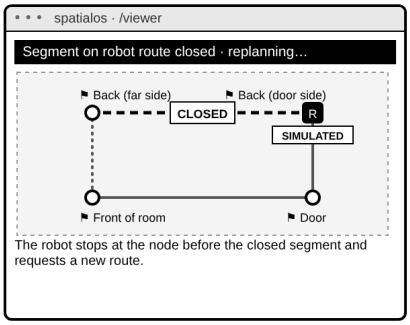
Detailed screens (graph drawn as a schematic of HSS 349, not to scale). The robot is simulated in the web app and is always labeled SIMULATED; it never represents a real robot.



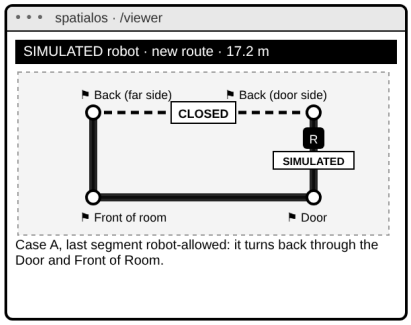
1 Choose the Robot profile. The route uses only open, robot-allowed segments (R5.4, built in M2).



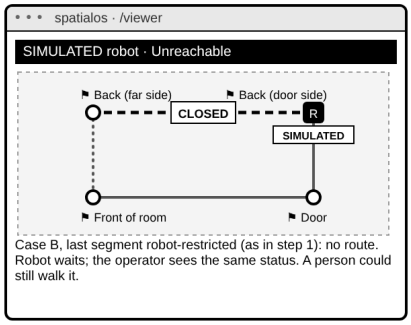
2 Start: the robot moves along the route as a black marker labeled SIMULATED (R10.1, R10.2).



3 An operator closes a segment ahead. The robot stops and replans (R10.3).



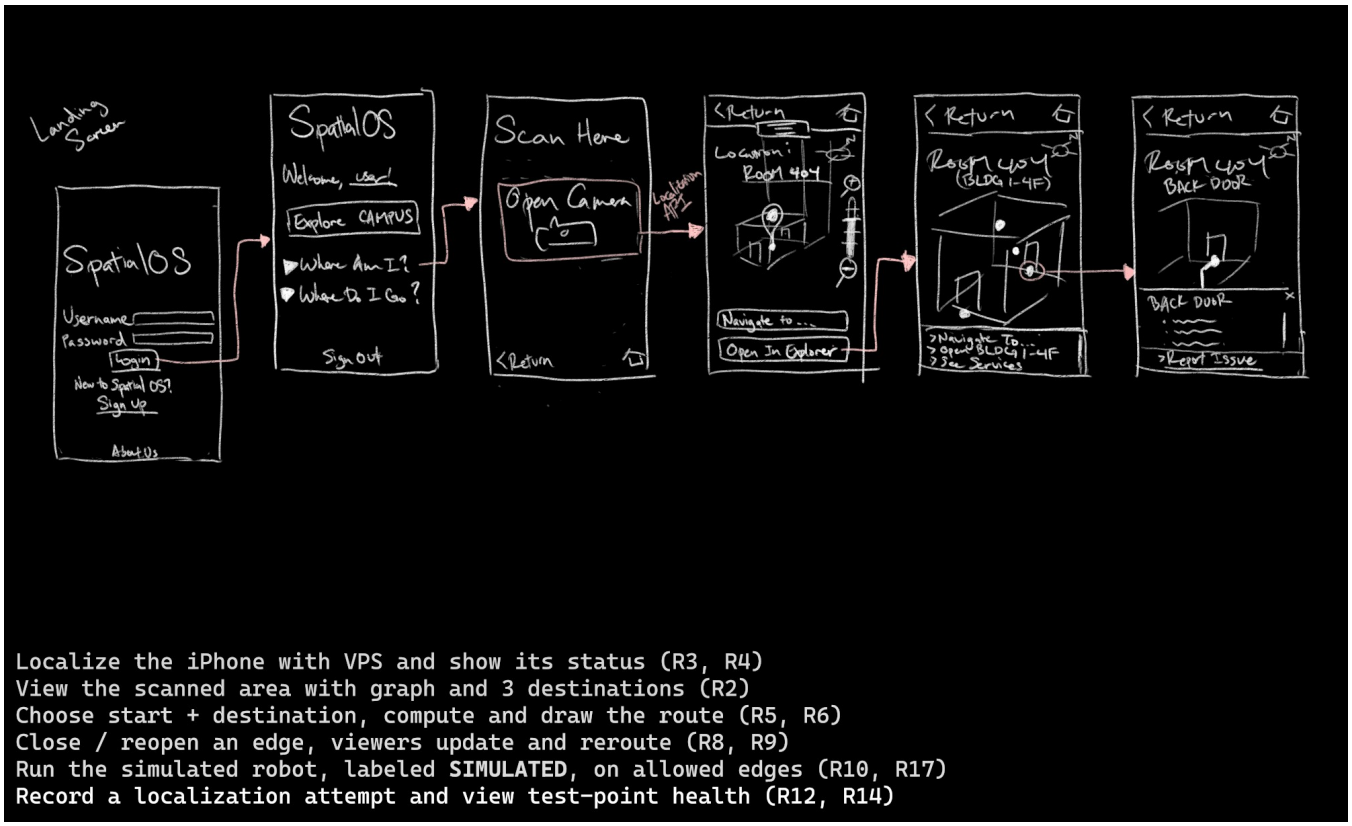
4 Replan succeeds: the robot follows the new route.



5 Replan fails: robot and operator both show "Unreachable" (R9.3).

Status: Robot route (R5.4) built in M2. Robot movement, label and replanning planned for M3 (R10.1–R10.3); pause and reset in M4 (R10.4–R10.5).

Flow 6: Record a localization attempt and view test-point health (R12, R14)

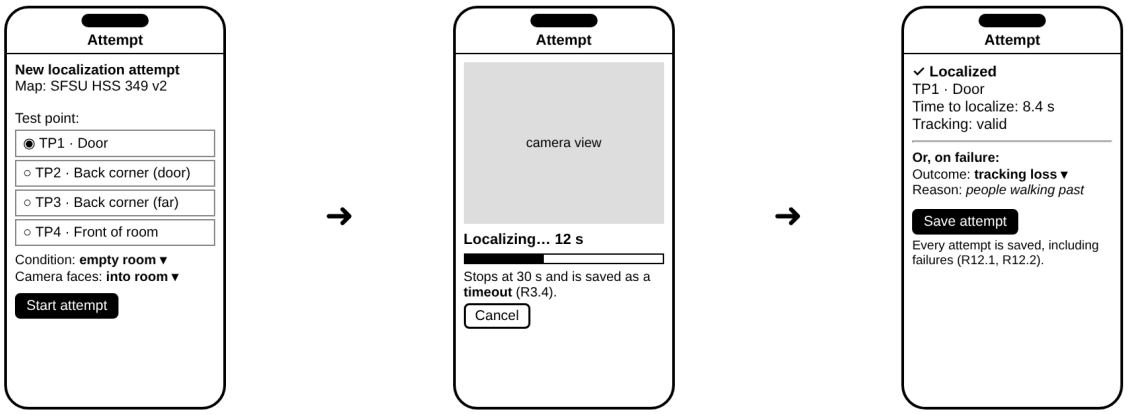



```
Localize the iPhone with VPS and show its status (R3, R4)
View the scanned area with graph and 3 destinations (R2)
Choose start + destination, compute and draw the route (R5, R6)
Close / reopen an edge, viewers update and reroute (R8, R9)
Run the simulated robot, labeled SIMULATED, on allowed edges (R10, R17)
Record a localization attempt and view test-point health (R12, R14)
```

Detailed screens, Flow 6

Flow 6 — Record a localization attempt and view test-point health (R12.1–R12.2, R3.4, R14.1–R14.5, R13, R16)

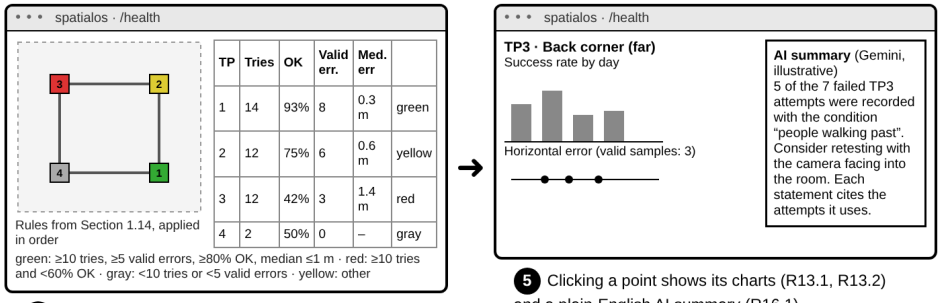
Detailed screens. A developer records attempts on the phone at fixed test points; the web dashboard turns them into per-point health. Numbers are example values.



1 On the phone, pick a test point and conditions, then start an attempt.

2 The phone tries to localize; a 30-second limit ends the attempt as a timeout.

3 Result is saved with its outcome and, for failures, a reason.



4 Dashboard: each test point is colored by the Section 1.14 rules from attempts, success rate, valid error samples and median error (R14.1–R14.5).

5 Clicking a point shows its charts (R13.1, R13.2) and a plain-English AI summary (R16.1).

Status: Planned. Attempt recording and timeout in M3 (R3.4, R12.1, R12.2); health, charts and AI summary in M4 (R13–R14, R16). M2 has only labeled synthetic attempts and the attempt search (R15.1).

3.2 Alignment with UX principles

These principles describe the intended design. Each point ends with its status: *Built (M2)*, *Planned* with its requirement ID, or *Proposed* (an idea not yet in Section 2).

Useful. The application serves to assist students and visitors in navigating around the campus. By using positional localization via VPS, the application aims to help users identify their position and route to a destination; the mobile-friendly application can route a path to a particular destination and identify current blockages/service disruptions. *Routing to a destination and live VPS position: Built (M2), R3, R5. Blockages: Planned, R8, R18.*

Usable. As a navigational tool, the application is designed with the mobile device in mind. Core functionalities are emphasized in larger buttons and additional information may be hidden behind minimized menus. Node and path information when navigating from one location to the other is collapsed as it is not necessary for routing, but is available if desired. Thus, these major functional requirements guide our emphasis on their corresponding UI interfaces. *Mobile-width /live page: Built (M2). Collapsed route details: Built (M2), route step list on /live.*

Desirable. The aesthetic appeal and desire of our app stems from simplicity and convenience. We reduce the amount of information on screen to only what is necessary and give users the agency to seek more detail if needed. Its flexibility in being able to compromise with users if they choose alternative routes makes our application stand out from other navigational tools. We also uniquely enable users to identify where they are using VPS. And by building off of the SFSU official branding – using parts of their color scheme and typeface – we assert its purpose as something that benefits our university. *Locating users with VPS: Built (M2), R3.1. Choosing alternative routes and SFSU branding: Proposed.*

Accessible. It is intended for navigators to decide whether they wish to take a particular path or not – the application allows them to choose whether to find alternative routes that avoid particular doorways and rooms, such as due to limited mobility

inhibiting their use of stairwells. Color-coded points of interest and paths are also taken into consideration regarding the 3D render, where differing components are designed to be distinguishable by shape and tone. *Labeled destinations and route on the 3D view: Built (M2), R2.4, R5.3. Routes that avoid stairs or doorways: Proposed.*

Findable. A significant problem that SpatialOS tackles is being able to find where someone may be on campus. By providing multiple means for locating where someone is, they would not be limited if we only provided VPS location and their device was unable to scan their environment. Searchability of rooms by name is a particular alternative. As for locating the functionality on our app, we intend on emphasizing the core location identification and routing while separating more specific details for if they interact with these points. We also hide details relevant to certain locations depending on their access privilege. Functionality like blockage resolution would therefore only be available to technicians and the like. *List of destinations: Built (M2), R15.3. Search of rooms by name: Proposed. Role-based access: Planned, R1.3–R1.4 (M3).*

Credible. When exploring our scanned 3D assets, we wish to provide a timestamp of the scans as well as allowing users to report inconsistencies in the visuals. Aggregating these reports, we can inform technicians to generate newer scans for up-to-date representations of reality. If time permits, we may also offer a cross-reference to the official SFSU site map to help users identify inconsistencies. *Scan timestamps, inconsistency reports and site-map cross-reference: Proposed.*

4. Architecture, Database and APIs

Author: Noah Bran, Backend Lead. **Reviewers:** Daksh Kumar, Ezeren Masangcay.

Revision (Oct 20): updated by Sumit Kumar to match the deployed code on `main` (89ad6c8) and Sections 1, 2 and 5.

Status: Built and deployed. The M2 schema, seed data and all four APIs run on production (`sfsu-csc648-06-fall26-team02.vercel.app`). Phone → production live position was verified in HSS 349 on Oct 20 (828 pose samples stored across 8 upload periods).

4.1 System architecture

SpatialOS has two clients that only talk to each other through our backend: the **Unity iPhone app** (localizes with Niantic VPS and sends its pose) and the **Next.js web app** (home page list/search, `/viewer` route planning on the 3D scan, `/live` navigation with the live phone dot). Supabase PostgreSQL 17 stores all data; Supabase Realtime pushes new poses to every open `/live` page.

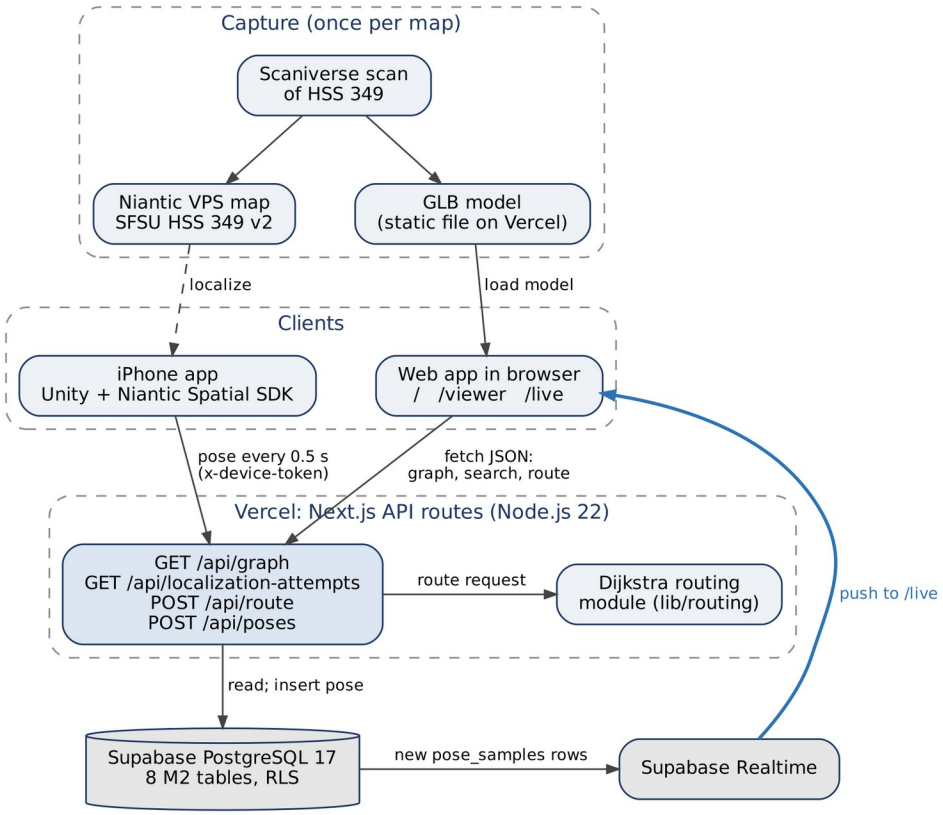


Figure 4.1: M2 system architecture. Diagram source: `diagrams/M2_Architecture.dot`.

Component	Technology	Role in M2
iPhone app	Unity 6 (6000.3.14f1), C#, Niantic Spatial SDK 4.1, iOS	Localizes against the VPS map; `PoseReporter`/`PoseUploader` send the pose every 0.5 s while tracking is valid
Web app	Next.js 16 App Router, React 19, TypeScript, Tailwind, Three.js 0.186	Home page list/search, `/viewer` (3D scan + graph + route), `/live` (2D map + live dot + route + arrival)
Backend	Next.js API routes on Vercel (Node.js 22)	Validation, routing, database access; no separate server
Routing	Team-written Dijkstra (`lib/routing`)	Shortest route for `human` or `robot` profile; unit tested with Vitest
Database	Supabase PostgreSQL 17 with row-level security	All structured data; Realtime publication on `pose_samples`

Component	Technology	Role in M2
3D model	Static GLB, `public/models/hss349-v2.glb` (4.9 MB)	Served by Vercel. Moves to Supabase Storage with the `visual_assets` table in M3 (Sections 1.12 and 5)

Spatial frame (no transform in M2). The HSS 349 world frame is the VPS map frame of *SFSU HSS 349 v2*. The GLB is exported from the same map, and the graph nodes were recorded with the iPhone while localized on it, so model, graph and poses already share one frame. The only conversion is handedness: Unity is left-handed, our world frame is right-handed and Y-up, so `/api/poses` stores `world = (x, y, −z)`. The viewer draws the graph at the floor height measured in the scan (−1.29 m). This conversion is implemented, but physical alignment is not yet validated: the walking direction (Door → Front) and position accuracy have not been checked against tape-measured reference points (planned M3, with the test points in R14). A `frame_transforms` table is needed only for maps whose model and VPS frame differ; it is designed in Section 1.12 and planned for M3.

Not built in M2 (designed, planned M3–M4): sign-in and roles, closures and rerouting, robot simulation, localization health and dashboard, Gemini summary, operator map editing.

4.2 Database organization

Migrations are in `app/backend/migrations/`: `202610110001_m2_foundation.sql` (7 tables), `pose_samples.sql`, `202610190001_hss349_four_destinations.sql`, `202610190002_pose_samples_data_origin.sql`, plus seeds `hss349_seed.sql` and `milestones/M2/seed.sql`. The M0 `users` and `team_members` tables are unchanged. Field-level definitions are in Section 1; table and field names here match it exactly.

Table	Main columns	Relationships
`sites`	`site_id`, `name`, `description`, `is_test_site`, `is_public_demo`, `active_map_version_id`	Has many map versions
`map_versions`	`map_version_id`, `site_id`, `version_label`, `publication_state`, `world_frame`, `vps_map_id`, `vps_status`, `current_graph_revision`, `current_path_state_revision`, `published_at`	Owns nodes, segments, destinations, test points, attempts, pose samples
`graph_nodes`	`node_id`, `map_version_id`, `graph_revision`, `label`, `position {x_m,y_m,z_m}`	Endpoints of path segments; anchor of destinations
`path_segments`	`segment_id`, `map_version_id`, `label`, `start_node_id`, `end_node_id`, `geometry`, `distance_m`, `direction`, `availability` (open/closed), `robot_access` (allowed/restricted)	Two nodes on the same map and revision
`destinations`	`destination_id`, `site_id`, `map_version_id`, `graph_revision`, `node_id`, `name`, `description`, `is_active`	One node
`test_points`	`test_point_id`, `map_version_id`, `label`, `purpose`, `reference_position`, `reference_method`, `reference_uncertainty_m`, `reference_is_verified`, `is_synthetic`	Has many attempts

Table	Main columns	Relationships
localization_attempts	attempt_id, map_version_id, test_point_id, device_id, sdk_version, mode, data_origin, condition_label, camera_direction, started_at, ended_at, outcome, latency_ms, failure_reason, horizontal_error_m, error_is_valid	One test point; may own pose samples
pose_samples	sample_id, map_version_id, attempt_id, device_id, provider_map_id, provider_pose, world_position, heading_deg, tracking_status (localizing/valid/lost), data_origin (real/synthetic), captured_at, received_at	One map version; optional attempt

Integrity rules enforced in the database. UUID primary keys. Composite foreign keys keep nodes, segments and destinations on the same map and graph revision. Checks reject unknown enum values, empty labels, non-finite positions, self-loops, segment geometry that does not start/end at its nodes, distance_m that differs from the geometry length by more than 0.01 m, and inconsistent attempt timing (success within 30 s with latency ≤ elapsed time; timeout ≥ 30 s; failures need a reason). pose_samples.data_origin only allows real or synthetic; the API (not the constraint) decides the value: poses from the fake-test device, or sent with data_origin: "synthetic", are stored as synthetic; all others as real. Referenced rows cannot be cascade-deleted.

Data in the database now.

Map	Content
SFSU HSS 349 (real scan, b2000000-…-0001)	4 nodes, 4 path segments (a loop around the room), 4 destinations: Door, Back Corner (Door Side), Back Corner (Far Side), Front of Room. Node positions are single phone readings, not tape-measured. Live pose samples from phone trials
Synthetic M2 demo (20000000-…-0001)	8 nodes, 10 path segments, 3 destinations (Room 103, Lobby, East room), 5 test points, 10 localization attempts, all labeled synthetic

4.3 Add / delete / search operations per table

M2 has no sign-in, so the only write path from the app is the phone's pose upload. Everything else is read-only in the app and seeded by migration.

Table	M2 operations (app)	Fields involved	Planned (milestone)
sites, map_versions	Display (only published maps of public demo sites)	name, version_label, world_frame, revisions	Administrator publishes a map version (role in Section 1.2) — M3
graph_nodes	Display; search by map (home page table, viewer, /live)	label, position	Operator adds a node by clicking the model (R7.1) — M4
path_segments	Display; search by map; used by routing	label, start/end node, distance_m, availability, robot_access	Closes/reopens (R8.1–R8.2), robot restriction (R17.1) — M3; operator connects nodes (R7.2) — M4
destinations	Display; select as route destination	name, node_id, is_active	Operator renames a destination (R7.3) — M5 if time
test_points	Display as search filter options	label	Developer adds test points — M3

Table	M2 operations (app)	Fields involved	Planned (milestone)
`localization_attempts`	Search by test point, outcome and point-label text; paging; total count	`test_point_id`, `outcome`, `q` (point label), `latency_ms`, `failure_reason`, `started_at`	Phone app adds attempts (R12.1–R12.2) — M3; CSV export (R15.2) — M5 if time
`pose_samples`	**Add** (phone, `POST /api/poses`); display latest pose and Realtime updates on `/live`	`world_position`, `heading_deg`, `tracking_status`, `captured_at`	Replay a recorded walk (R11.1–R11.2) — M5 if time

Routes are computed on request and not stored in M2. No table has a delete operation in the app; corrections are made by reviewed SQL.

4.4 Backend APIs

Full request/response specification: `milestones/M2/CONTRACTS.md`; shared TypeScript types: `app/frontend/lib/spatial/contracts.ts`. All four endpoints are deployed and used by the production pages.

Endpoint	Behavior	Used by
`GET /api/graph`	Query: `map_version_id`. Returns site, frame, revisions, nodes, path segments, active destinations and synthetic test point options from one database snapshot	Home page, `/viewer`, `/live`
`GET /api/localization-attempts`	Query: `map_version_id` (required), `test_point_id`, `outcome`, `q`, `limit`, `offset`. Lists synthetic attempts; filters combine with AND (test point, outcome, point-label substring `q`); `limit` 1–100, `offset`; returns `items` and `total`	Home page attempt search
`POST /api/route`	Body: `map_version_id`, `start_node_id`, `destination_id`, `profile` (`human`/`robot`). Loads the graph, runs Dijkstra, checks the traversal, returns ordered steps and total distance; unreachable → empty steps and null distance; start = destination → empty route, 0 m	`/viewer`, `/live`
`POST /api/poses`	Header `x-device-token`. Body: `sample_id`, `map_version_id`, `device_id`, `provider_pose` (position + quaternion), `tracking_status`, `captured_at`, optional `provider_map_id`, `attempt_id`, `data_origin`. Validates, checks the map is published, converts to world (z flip), computes `heading_deg`, inserts into `pose_samples`. New sample → 201; retry of the same `sample_id` → 200 (idempotent, not stored twice). Response: `sample_id`, `world_position`	iPhone app

Errors. Graph/search/route: malformed input → 400, unknown or private entity → 404 (without revealing that a private one exists), no search match → 200 with empty `items` and `total: 0`, database failure → sanitized 503, routing produced an invalid result → 502 (`INVALID_GRAPH` if the stored graph cannot be routed, `INVALID_ROUTING_RESULT` if the returned path fails the traversal check). Poses: bad token → 401, invalid body → 422 with a reason, server error → 500. Responses are not cached, so database edits show on refresh.

Live position flow. Phone localizes → `POST /api/poses` every 0.5 s → row in `pose_samples` → Supabase Realtime → every open `/live` page moves the dot. `/live` treats the position as live only when tracking is valid and the pose is under 5 s old, picks the nearest node as the route start (0.75 m hysteresis), and shows arrival within 1.0 m of the destination. Sequence diagram: Section 5, Flow 7.

4.5 Access control and credentials

Actor	Access in M2	How it is enforced
Anyone with the site URL	Read published public demo maps, their graph, destinations and synthetic attempts; request routes	Read APIs use the anon/publishable key only, and RLS allows SELECT only on published maps of sites with `is_public_demo = true`; real trial records stay hidden
Browser reading poses (`/live`)	Reads `pose_samples` directly with the anon key and subscribes to Realtime	**Known gap:** the current policy allows anon to read every pose sample. Restricting it to public demo maps is planned for M3. Pose samples contain a device label and timestamped position/orientation data; no names or email addresses are stored
iPhone app	Insert pose samples through `POST /api/poses` only	Shared `DEVICE_TOKEN` (Vercel env var, constant-time comparison); this one route uses the server-only service-role key to insert. No other write endpoint exists
Team, TA and instructor	Full access in the Supabase dashboard	Supabase project members (invited in M0)

No keys or tokens are in the repository: Supabase keys and `DEVICE_TOKEN` live in Vercel environment variables, and the phone's `uploader.local.json` stays outside the repo. The device token will be rotated after M2 grading. Sign-in and the visitor/developer/operator/administrator roles (R1.1–R1.4) are designed in Section 1.2 and planned for M3.

4.6 Third-party APIs, open-source components and tool changes

Component	Use	License / note
Niantic Spatial SDK 4.1 (Unity)	VPS localization on the iPhone	Niantic terms; SFSU org account via TA
Scaniverse	Scan capture, VPS map generation, GLB export	Niantic; shared org credits
Three.js 0.186 + `GLTFLoader`, `OrbitControls`	3D viewer	MIT
`@supabase/supabase-js`	Database queries and Realtime	MIT
Next.js, React, Tailwind	Web app	MIT
Vitest	Unit tests (routing, validation)	MIT

Changes since M0/M1: phone platform is iPhone (Unity + NSDK) instead of Android; Unity, Three.js and Vitest added to the approved stack (TA approval for these additions should be confirmed); the GLB is served by Vercel in M2 (Supabase Storage from M3); Gemini is not used until M4.

4.7 Validation and remaining work

Verified. Typecheck, lint and 74 tests passed before the final M2 merges (PR #43 → `dev`, PR #44 → `main`, Oct 19). SQL tests cover cross-map/revision constraints, geometry, timing and anon access; API tests cover filters, paging, validation, empty results, error sanitization and routing (human, robot, unreachable, zero-distance). On production (Oct 19) `/viewer` loads the HSS 349 scan with 4 destinations and returns Door → Front of Room = 5.1 m. Phone → production (Oct 20): 828 pose samples reached production across 8 upload periods separated by gaps of more than 60 s, 4 before class (12:33–12:59) and 4 during the in-class demo (14:03–14:40); the live dot updated on `/live` on other phones and browsers. Upload-period table, query and screenshot: `evidence/live-position.md`. Evidence: `milestones/M2/evidence/`.

Remaining (M3). Limit pose reads to public demo maps; enforce R6.4 (no live start from a stale or lost pose); fix HSS 349 map metadata (still v1 seed values); move `/api/poses` to `sfo1` if the dot lags; use `active_map_version_id` instead of hardcoded map IDs; re-measure nodes.

5. High-Level UML Diagrams

This section shows the SpatialOS design in UML and how much of it is built for M2. Section 5.1 gives the class diagrams: the data classes used in M2 (Figure 5.1), the live localization and health classes (Figure 5.2a), the classes planned for M3 to M5 (Figure 5.2b), and the code units that work on them (Figures 5.3a and 5.3b). Section 5.2 gives sequence diagrams for the seven major flows. Flows 1 to 6 follow the full system design; Flow 7 shows the live position on the route, which is part of the M2 demo. Section 5.3 maps each flow to its requirements, Section 5.4 lists the main design decisions, and Section 5.5 states which code exists today.

Names match Section 1 (tables, fields, enums) and the code. Scope tags also follow Section 1: **[M2]** is built for this milestone, **[M2 partial]** is partly built, and **[Planned]** is built in M3 to M5. A requirement scheduled for a later milestone that is already working is marked *built early in M2*. In the sequence diagrams, gray shaded blocks mark later-milestone steps, and each block is labeled with its milestone.

Coordinate frame in M2. The world frame is the VPS map frame of *SFSU HSS 349 v2*: right-handed, Y up, meters. The scanned mesh was exported from this map and the four graph nodes were recorded with the phone while localized on it, so the model, the graph and the live poses line up without a stored transform. The server only flips Z, because Unity is left-handed.

5.1 Class diagrams

Five figures follow, one per page. What to notice in each:

Figure 5.1:

- A Site has map versions. A MapVersion owns its graph nodes, path segments, destinations and test points, and linked records must share one map version and graph revision.
- A PathSegment has two separate links to GraphNode: start node and end node. Its direction field says whether it can also be walked from end to start.
- Route is an API response in M2 and a stored routes table from M3. Each RouteStep gives the travel direction explicitly (from_node_id to to_node_id).
- The HSS 349 map has 4 graph nodes, 4 path segments and 4 destinations: Door, Back Corner (Door Side), Back Corner (Far Side) and Front of Room.
- In M2, LocalizationAttempt rows are labeled sample records used for list and search. The phone starts logging attempts in M3.
- In M2 the 3D model is a static GLB file served by Vercel (`public/models/hss349-v2.glb`). It moves to Supabase Storage and the visual_assets table in M3.

Figure 5.2a:

- PoseSample is built in M2. The pose_samples table stores every pose the phone sends through POST /api/poses, with the raw provider_pose and the converted world_position. Poses from the fake-test device are stored with data_origin = synthetic.
- From M3, a PoseSample belongs to a NavigationSession and records the transform used to convert it, and a successful live attempt links its pose through valid_pose_sample_id.
- FrameTransform is not needed for HSS 349 because the VPS frame is the world frame (Section 5 introduction). It becomes a table in M3, when other maps are added, and live navigation then uses only verified transforms.
- MetricSnapshot stores the exact attempt_ids it was calculated from, and an AiSummary explains one snapshot.

Figure 5.2b:

- SimulatedObstacle, AgentReport and PathEvent form the blockage chain. A tester places an obstacle, the robot reports it, and only an operator's confirmation closes the segment, through exactly one PathEvent.
- Every PathEvent records who made the change. Roles are per site through SiteMembership.

Figure 5.3a:

- Clients write only through API route handlers, which validate every request. The only direct database access from a browser is read only: /live reads the latest pose and subscribes to new poses through Supabase Realtime. Row-level security allows reads only; in M2 the pose policy still allows reading every pose, and limiting it to public demo maps is planned for M3.
- RoutePlanner (/viewer), LivePage (/live) and, from M3, the robot simulation all call RouteService, so a person, a live walker and a robot get routes from the same algorithm.
- The iPhone app sends its pose every 0.5 s through PoseUploader. Attempt logging (LocalizationClient) follows in M3.
- M2 has no sign-in. POST /api/poses is protected by a shared device token, and the read endpoints serve only the public demo map. Site roles arrive with RoleGuard in M3.

Figure 5.3b:

- Route handlers do input and output only. Request parsing is in validation, database access is in SupabaseRepository, and routing is in the pure RoutingEngine, so each part can be tested on its own.
- RouteService loads the graph, calls findRoute through the routing adapter, and checks every returned step against the database graph (validateRoutingResult) before answering. A wrong result is never sent to a viewer.
- PoseService decides data_origin on the server with poseOrigin: poses from the fake-test device, or marked synthetic by the client, are stored as synthetic. Poses from any other device are stored as real, so test builds must use the fake-test device or send data_origin = synthetic.
- HealthService will load attempts through SupabaseRepository, pass them to HealthCalculator and save the snapshot; the calculator never touches the database. RoleGuard will be the single place where site roles are checked.

Figure 5.1: M2 core data classes (selected core attributes)

The classes behind the M2 prototype: the scanned site, its graph, destinations, test points, sample attempts and the route response.

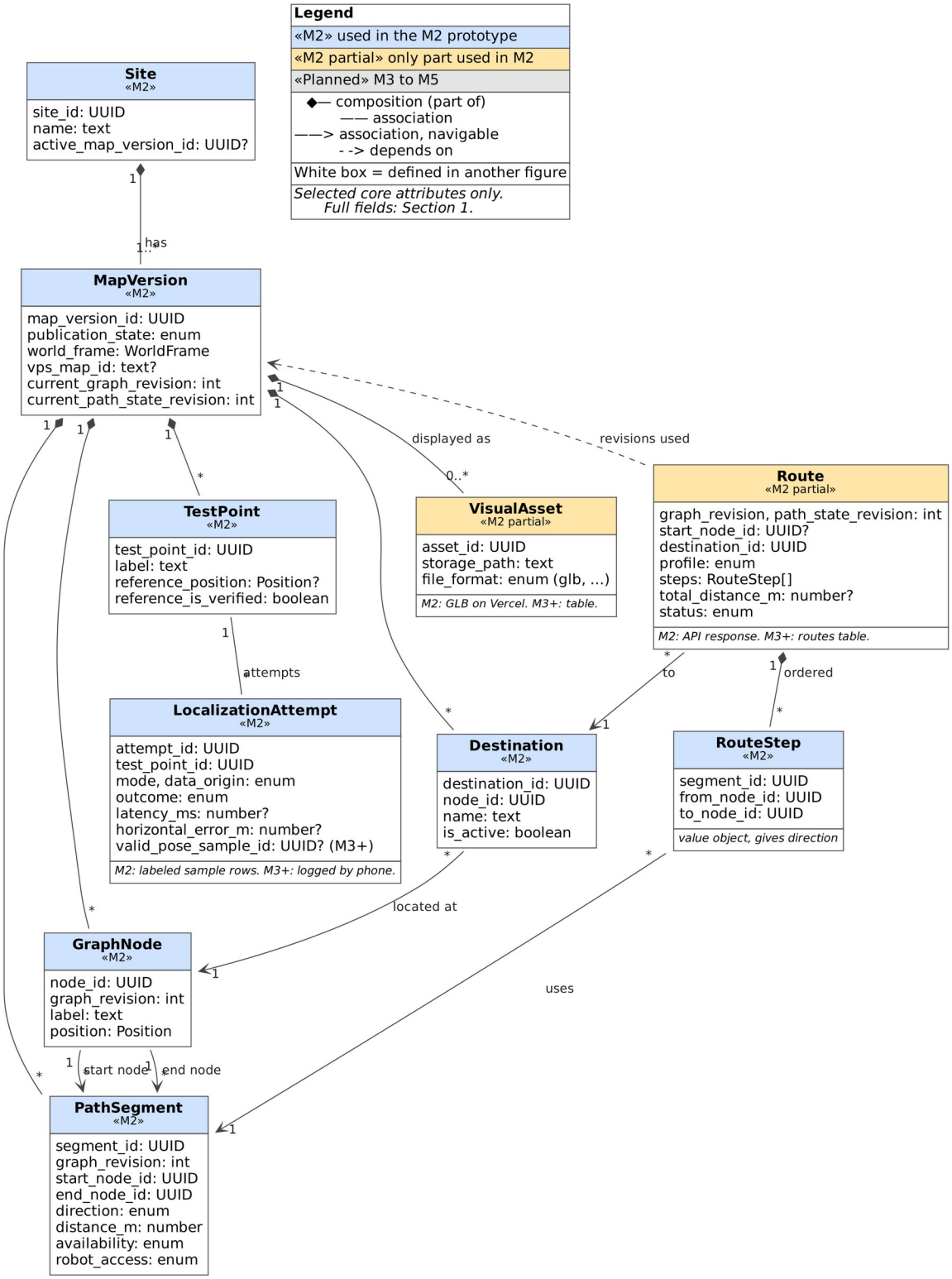


Figure 5.1: M2 core data classes (selected core attributes)

Figure 5.2a: Live localization and health data classes (M2 partial to M4)

Raw and converted poses, sessions, coordinate transforms and the health snapshot. White boxes are defined in Figure 5.1.

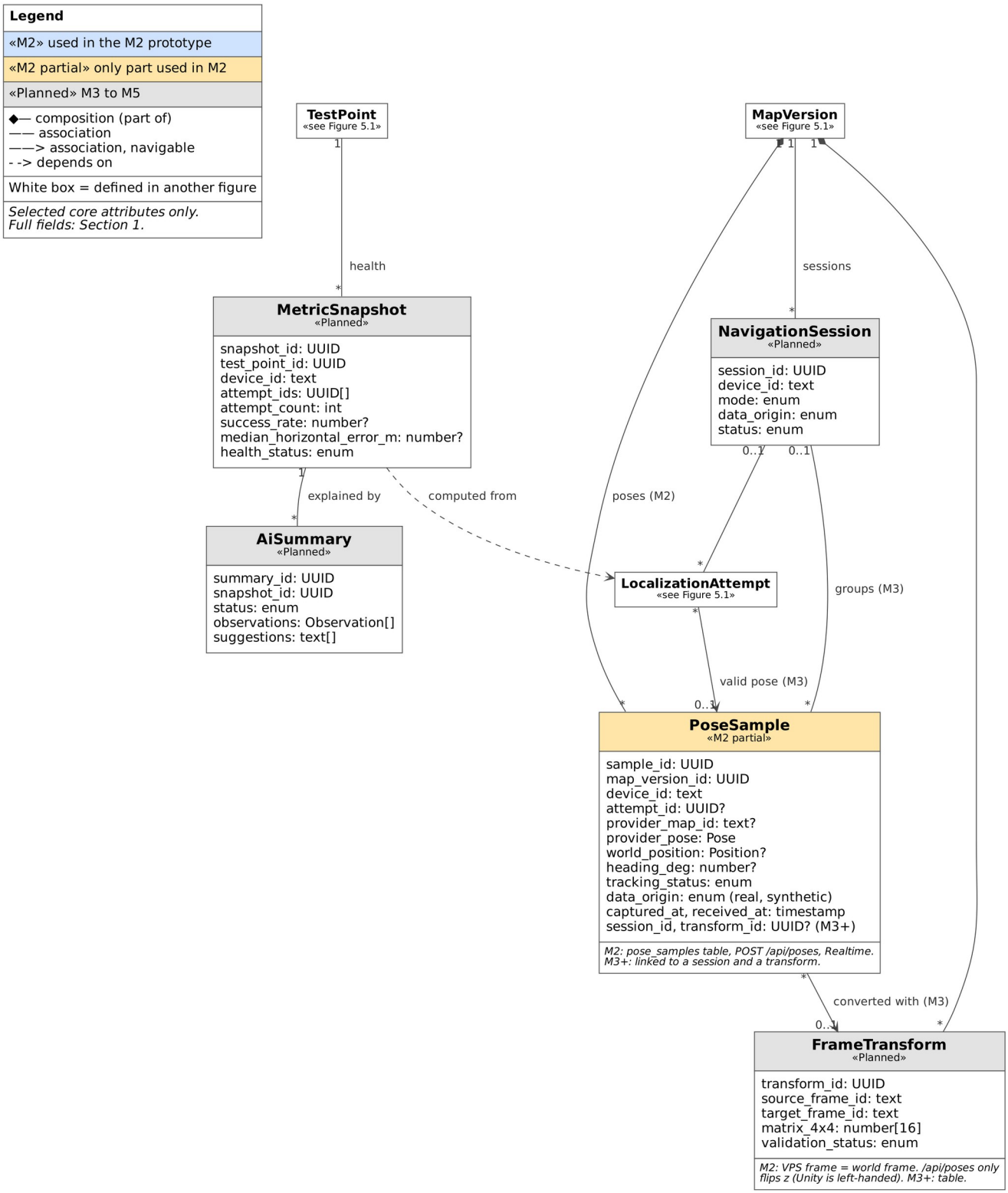


Figure 5.2a: Live localization and health data classes (M2 partial to M4)

Figure 5.2b: Planned data classes: simulation, path history and roles (M3 to M5)

The simulated robot, obstacles, operator review, path history and site roles.

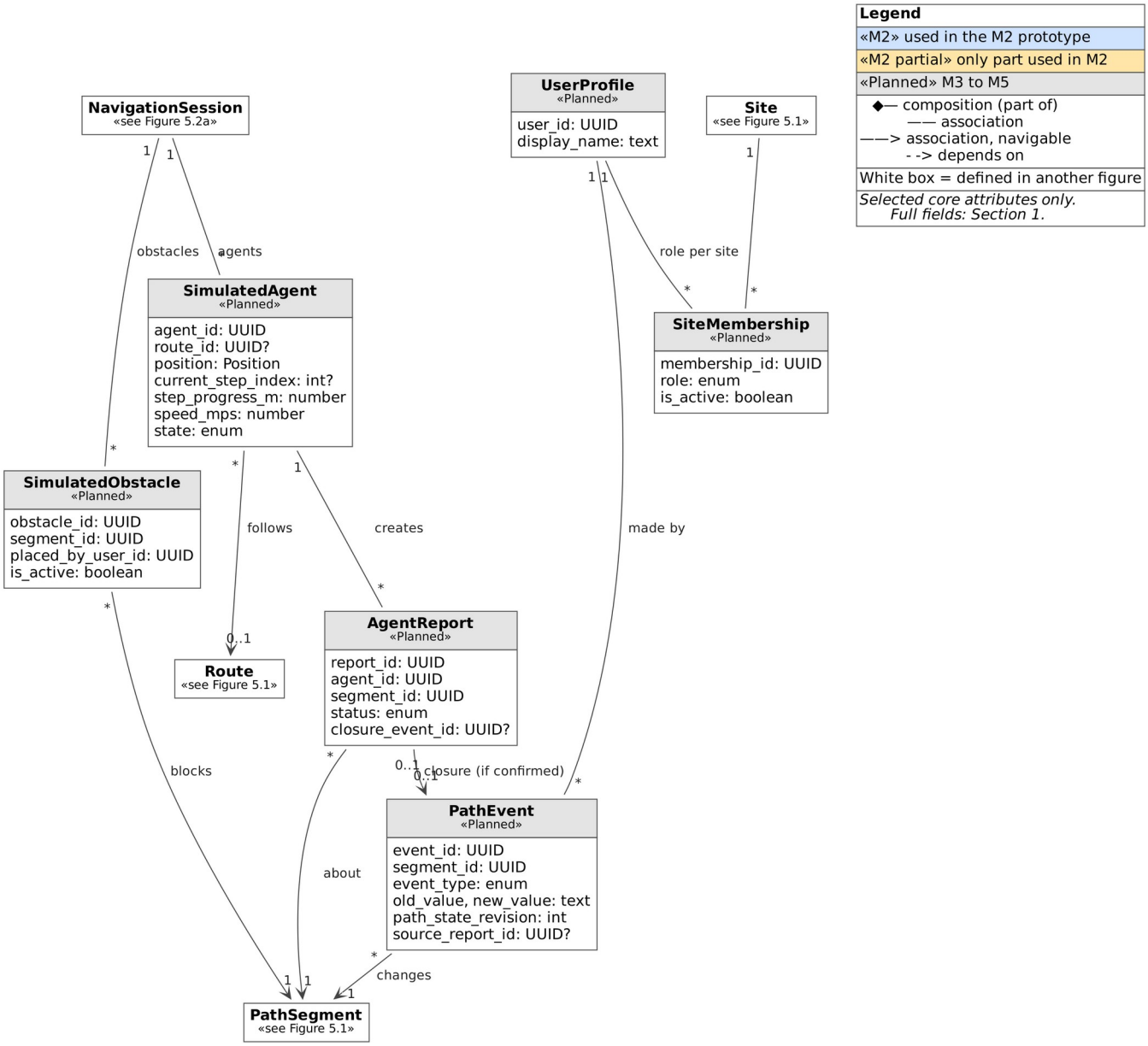


Figure 5.2b: Planned data classes: simulation, path history and roles (M3 to M5)

Figure 5.3a: Implementation units: clients and the API endpoints they call

Each box is tagged with its kind of unit (class, function module, React component, API route handler or Unity script) and whether it exists in the repository or is proposed.

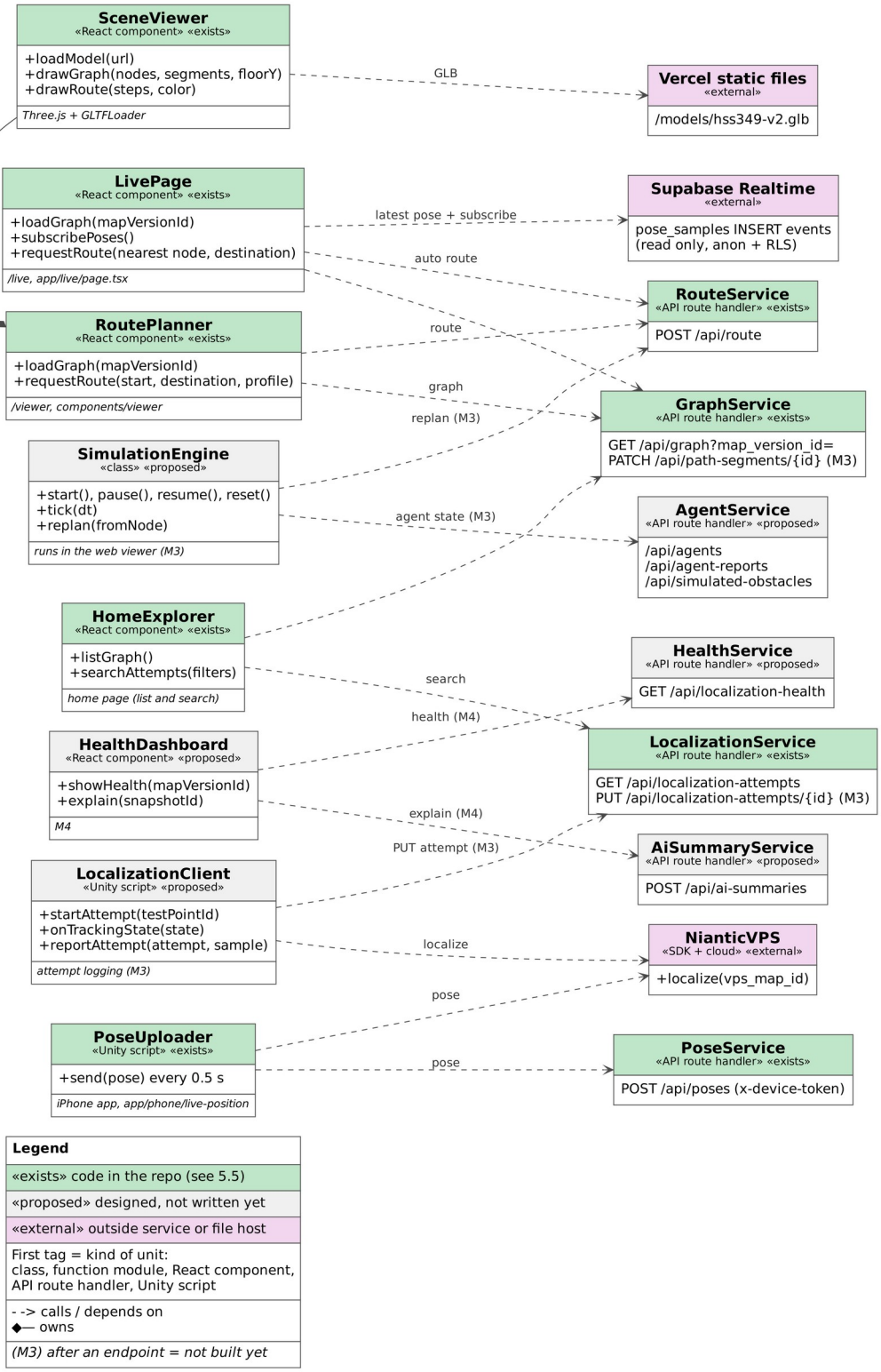


Figure 5.3a: Implementation units: clients and the API endpoints they call

Figure 5.3b: Implementation units: API handlers and core logic

What each API route handler does and what it depends on.

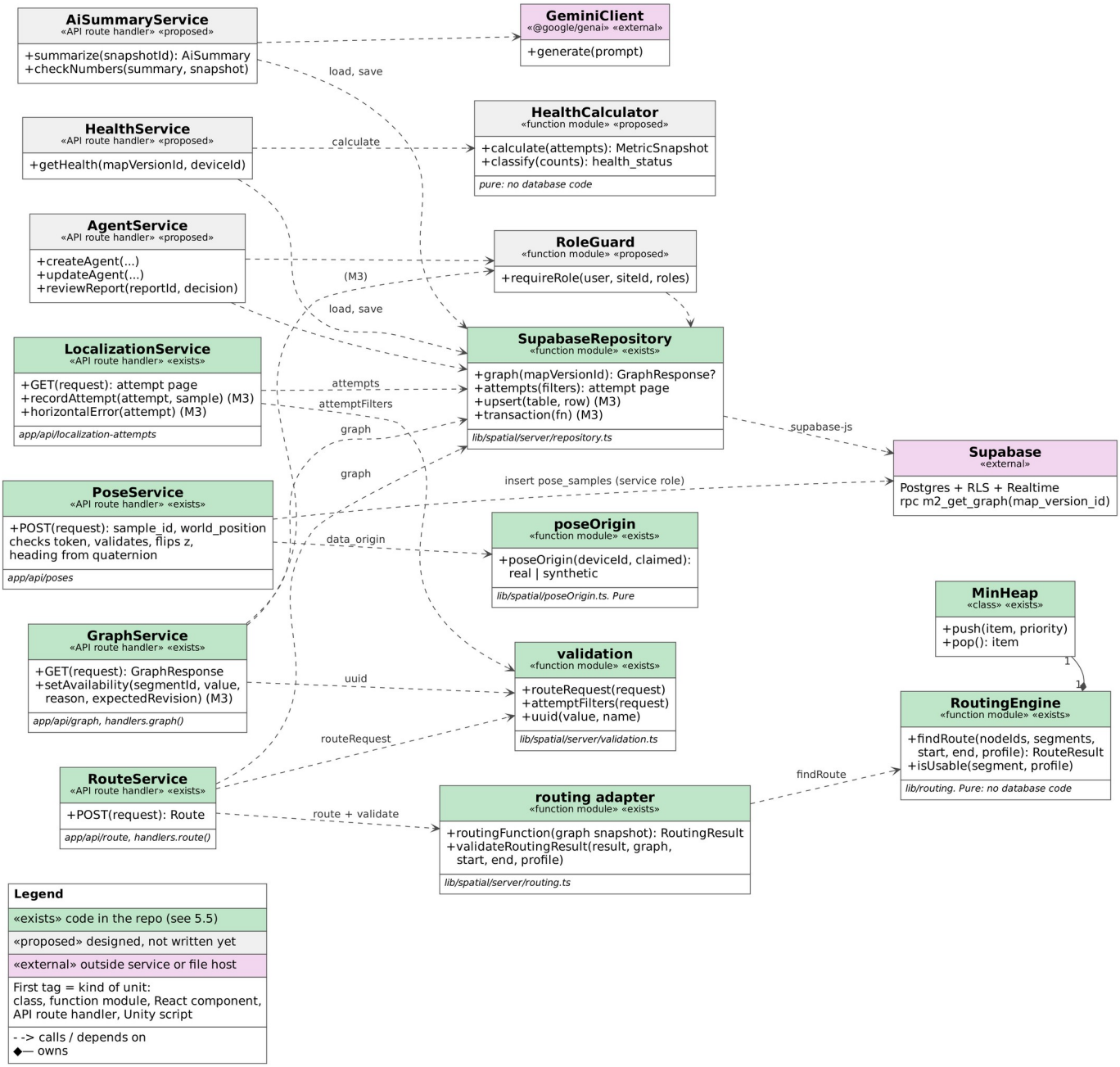


Figure 5.3b: Implementation units: API handlers and core logic

5.2 Sequence diagrams

Each flow lists its requirements and what is built in M2. Gray shaded blocks inside a diagram are later-milestone steps.

Flow 1: Localize the iPhone with VPS and show its status

Purpose: A developer stands at a test point and the iPhone localizes against the Niantic VPS map of the site.

Requirements: R3.1 (M2); R3.4, R4.1, R12.1, R12.2 (M3). Pose upload (R3.2) and the stale rule (R4.2) are built early in M2 and shown in Flow 7.

Scope: [M2 partial] M2: localization on the phone and its result (R3.1). M3: attempt records, timeout record and tracking states from the phone.

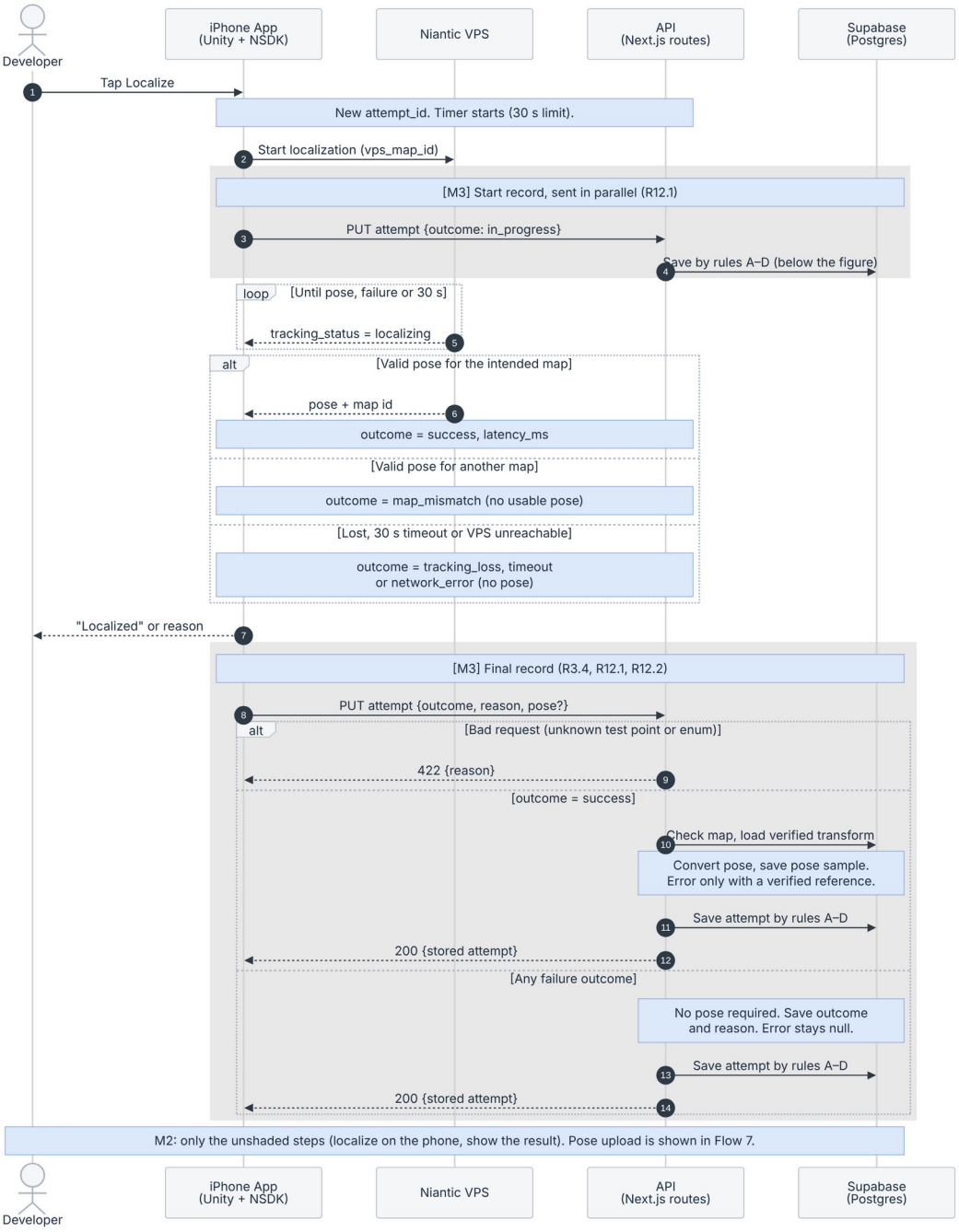


Figure 5.4: Localize the iPhone with VPS

What to notice

- The timer runs from the start of localization to the first valid pose. From M3 the start record is sent in parallel, so it never delays localization.
- Success means a valid pose for the intended map within 30 seconds. The SDK also reports its own tracking state (for example PRECISE); that is the SDK's confidence label, not a measured accuracy. Accuracy is measured separately at verified test points.
- Failures are always saved. map_mismatch, tracking_loss, timeout and network_error need no usable pose; the server stores the outcome and the reason, and horizontal error stays null (R12.2).
- Successes are checked first: the test point belongs to the map version, the VPS map matches its vps_map_id, and a verified transform exists. The phone sends the raw pose; the server converts it and calculates horizontal error only when the test point has a verified reference. Clients therefore cannot choose the stored error value.
- A pose older than 5 seconds, or with tracking that is not valid, cannot start a new live route (R4.2, R6.4, M3). In M2, /live already shows such a position as "Not live".

Save rules for an attempt (one atomic upsert on attempt_id)

A. No record exists: create it with the outcome received, finished or in_progress.

B. Stored outcome is in_progress and a finished outcome arrives: finish it.

C. The same finished outcome arrives again (a retry): change nothing and return the stored attempt.

D. An in_progress message arrives after the attempt is finished: change nothing. A finished attempt never goes back to in_progress.

The result is kept even if the start message is lost or arrives late, and retries never create duplicates. Pose samples are deduplicated by sample_id: resending the same sample does not create another record.

Flow 2: View the scanned area with its graph and four destinations

Purpose: A visitor opens /viewer and sees the scanned 3D model of HSS 349 with the route graph and the four labeled destinations.

Requirements: R2.2, R2.3, R2.4, R15.3 (M2). HSS 349 has four destinations, one per node.

Scope: [M2] Fully in M2. From M3 the active map comes from the site record and the model location from the visual_assets table.

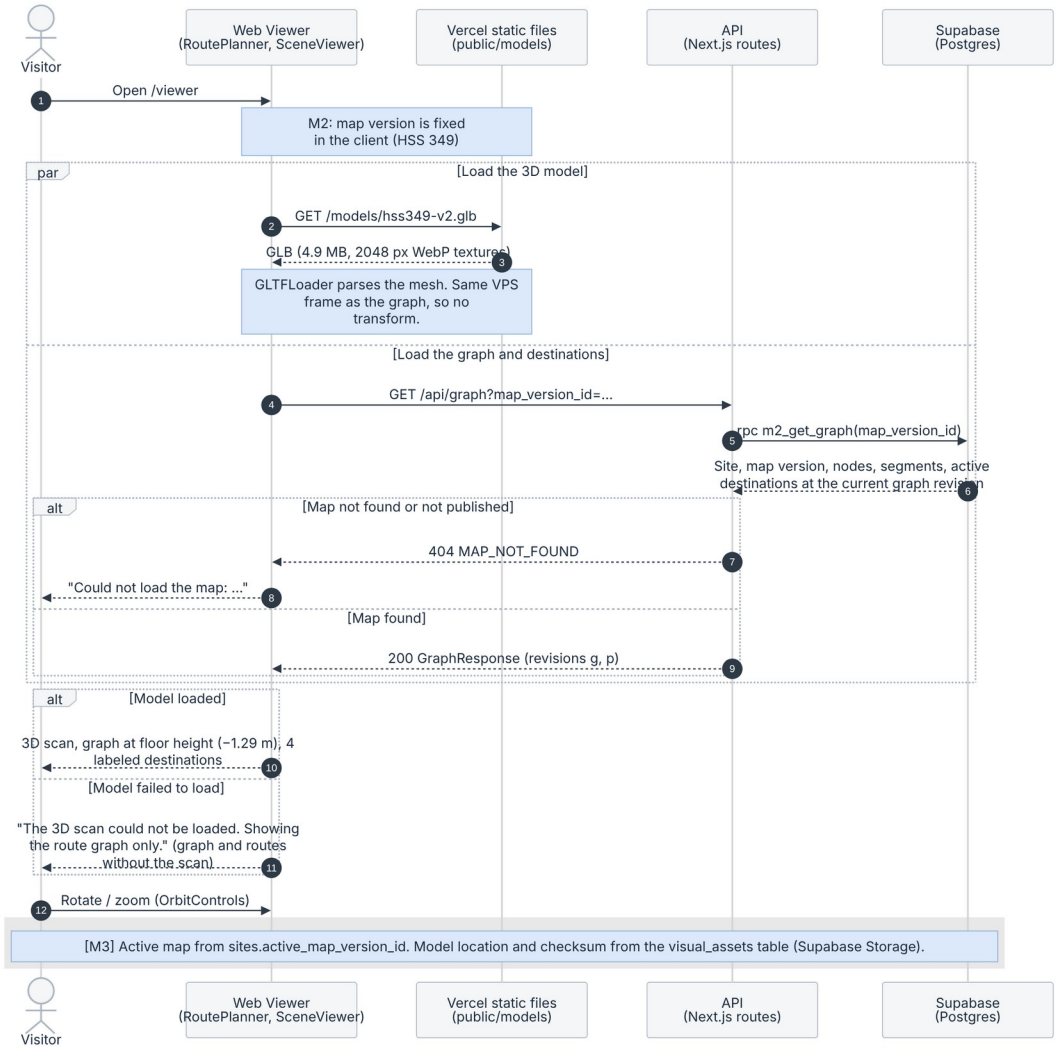


Figure 5.5: Load the scanned area, graph and destinations

What to notice

- The viewer loads the GLB from Vercel and the graph from GET /api/graph at the same time. In M2 the map version is fixed in the client; from M3 it is read from sites.active_map_version_id.
- The graph comes from one database function, m2_get_graph. It returns only a published, public map version, the rows of its current graph revision, and active destinations.
- The mesh shares the frame of the graph nodes, so no transform is applied. Nodes were recorded at phone height (y = 0), so the viewer draws the graph at the measured floor height of −1.29 m.
- If the model fails to load, the page shows an error message and still draws the graph, destinations and routes without the scan, so routing can still be shown. (A grid floor is drawn only when no model is configured.)
- Textures were reduced from 3072 to 2048 px WebP (14.5 MB to 4.9 MB) so the page loads quickly and stays within phone GPU memory.

Flow 3: Choose a start and destination, compute and draw the route

Purpose: A visitor picks a start and a destination by hand, chooses Person or Robot, and the route is drawn on the model.

Requirements: R5.1, R5.2, R5.3, R6.1 (M2); R5.4 robot route (M3) and R5.5 route distance (M4), both built early in M2.

Scope: [M2] Manual-start route for a person or the simulated robot, with its distance. M3: revision check on route requests and stored routes.

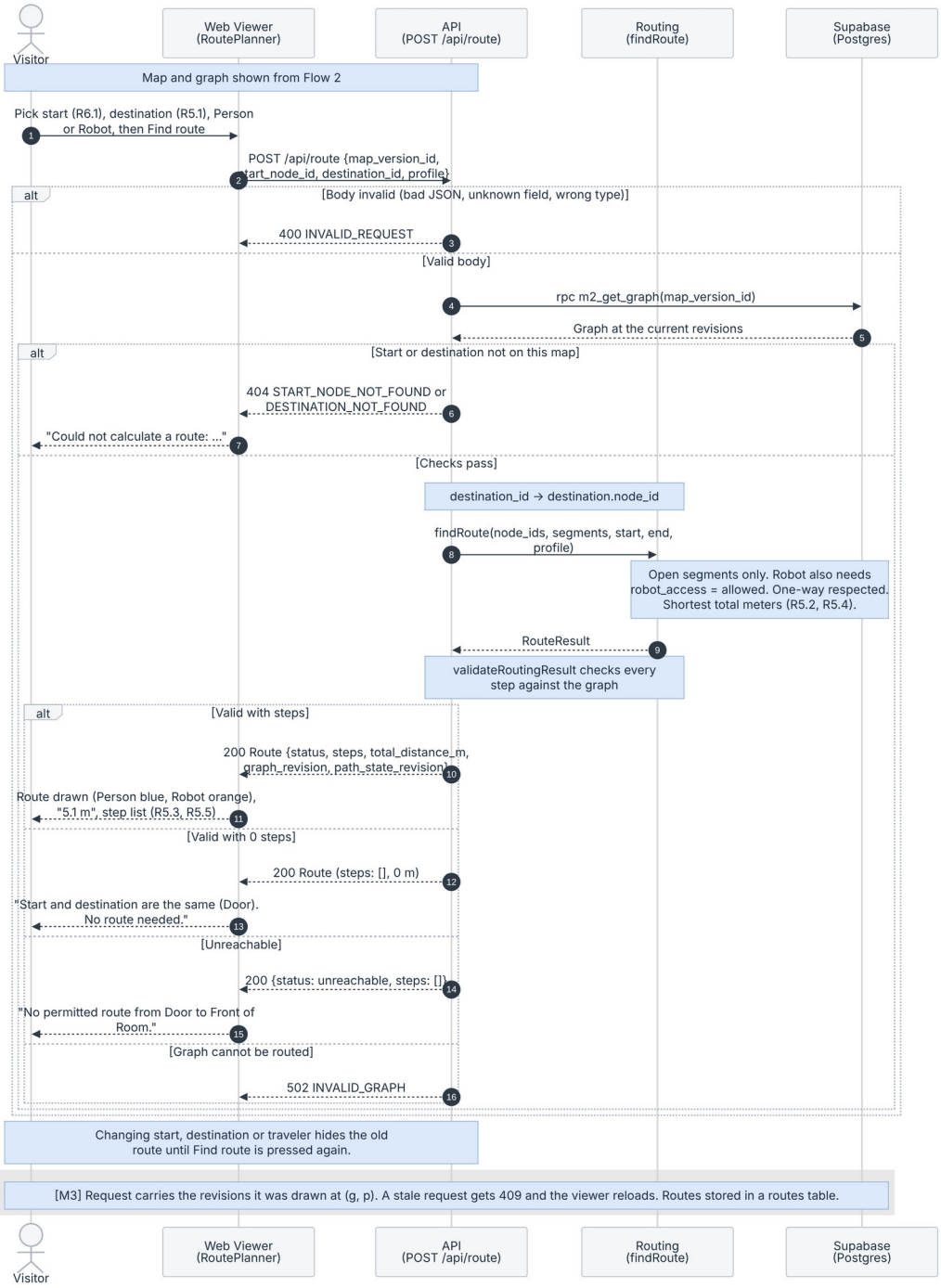


Figure 5.6: Route request

What to notice

- The API checks the request body first (400), then that the start node and destination belong to the map (404), then resolves destination_id to the destination's node before calling findRoute.
- findRoute uses open segments only, walks start_to_end segments in one direction only, and returns the shortest total distance (Dijkstra). For the robot it also skips segments with robot_access = restricted.
- validateRoutingResult checks every step against the database graph: connected, open, allowed for the profile, and no repeated node. A wrong result returns 502 and is never drawn.
- Results: valid (route drawn with its distance); valid with zero steps when start and destination are the same ("No route needed", 0 m); unreachable ("No permitted route"). invalid_input is an API error and is never stored as a Route.status.
- Changing the start, destination or traveler hides the old route until Find route is pressed again, so a person's route is never shown as the robot's.
- Distances are shown with one decimal (5.1 m) on /viewer and /live; the database keeps millimeter values. The response already carries graph_revision and path_state_revision; from M3 the request also sends the revisions it was drawn at, and a stale request gets 409 (decision 2).

Flow 4: Close or reopen a path segment, viewers update and reroute

Purpose: An operator closes a segment (for example, a wet floor) and every open route that used it is replanned.

Requirements: R8.1, R8.2, R8.3, R9.1, R9.3 (M3).

Scope: [Planned] M3. In M2, availability is seeded and can only be changed by hand in Supabase.

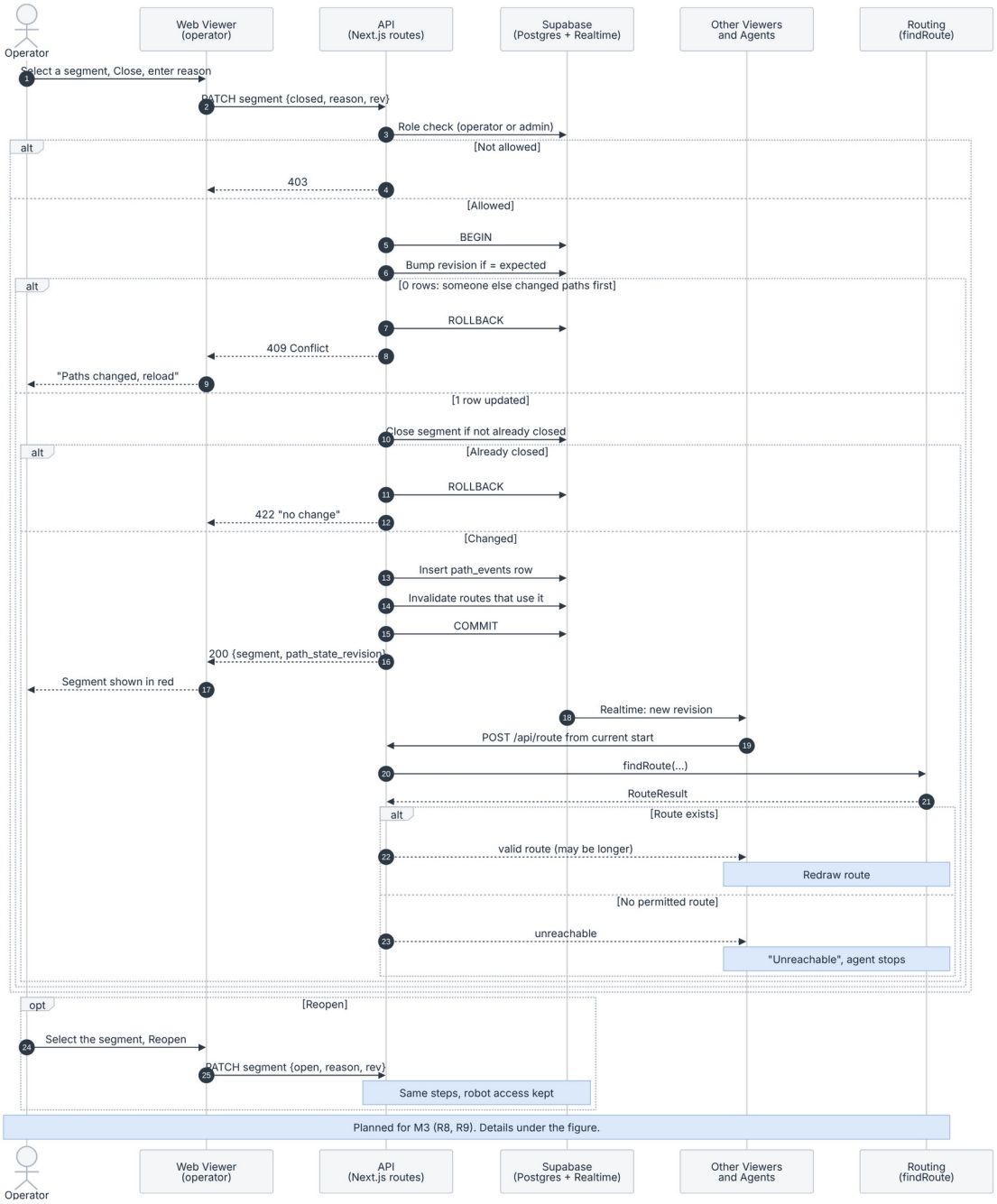


Figure 5.7: Path closure and rerouting

What to notice

- Step 6 is one conditional update: `UPDATE map_versions SET current_path_state_revision = expected + 1 WHERE map_version_id = … AND current_path_state_revision = expected`. If no row changes, another operator was first; the transaction rolls back and the client gets 409 Conflict.
- Step 10 changes the segment only if it is not already closed; otherwise the request gets 422 “no change”.

- The revision bump, the segment change, the path_events row and the invalidation of stored routes that use the segment commit in one transaction, so history never disagrees with the current state.
- Viewers and agents replan from their current routing start: a viewer from its chosen start node, a moving robot from its replan node (Figure 5.9). Reopening uses the same steps and keeps the segment's robot access (R17.2).

Flow 5: Run the simulated robot on allowed segments

Purpose: A tester starts a robot labeled SIMULATED. It follows a robot route, reacts to path changes and reports blockages to an operator. Figure 5.8 shows the run loop and controls, and Figure 5.9 shows the events checked on every tick.

Requirements: R10.1, R10.2, R10.3, R9.2 (M3); R10.4, R10.5, R18.1–R18.5 (M4); R18.6 (M5). The robot route profile itself (R5.4) is already built (Flow 3).

Scope: [Planned] M3: start, move and replan. M4: pause, reset, obstacles and operator confirmation. M5: operator rejection.

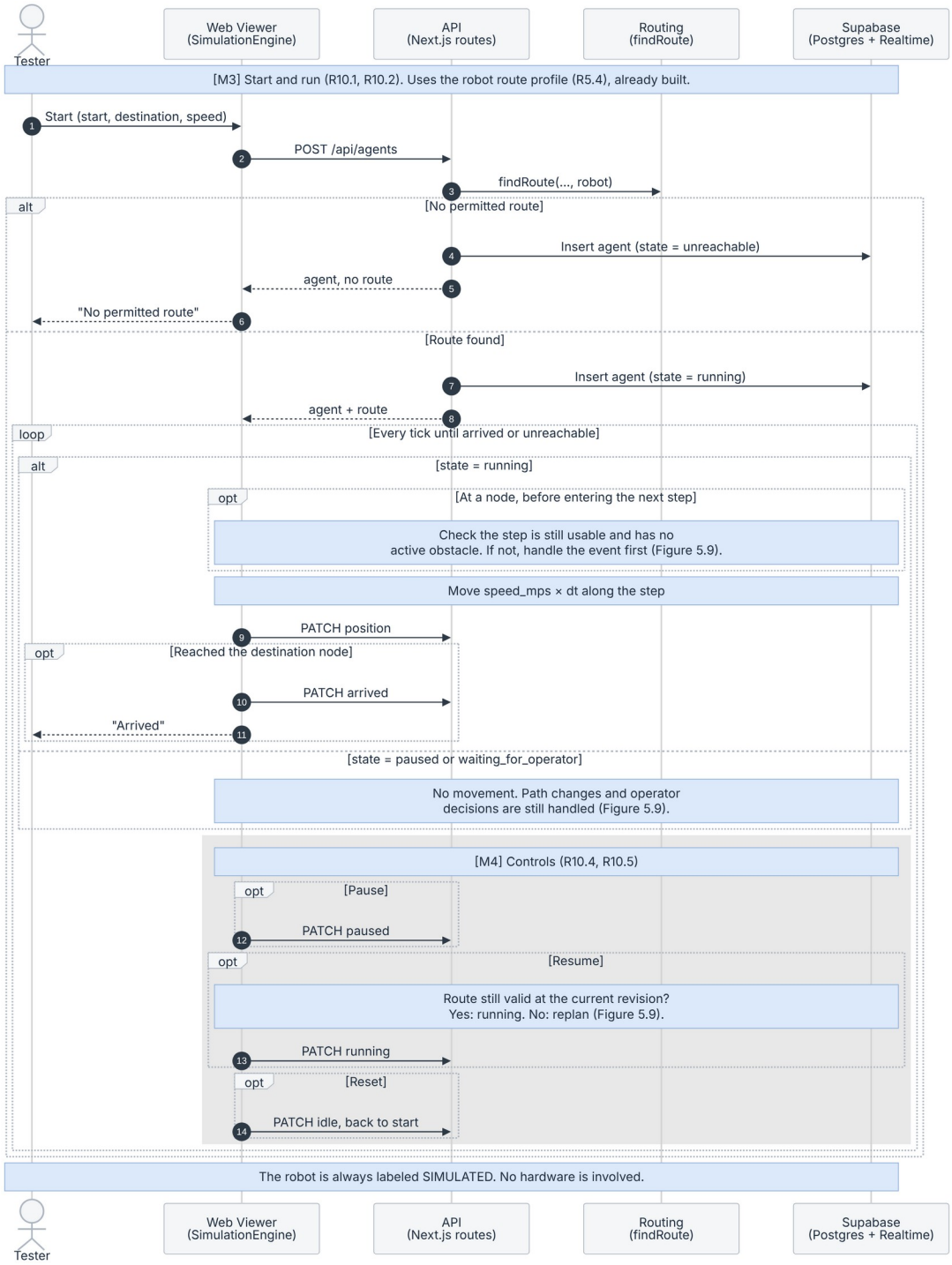


Figure 5.8: Simulated robot, run loop and controls

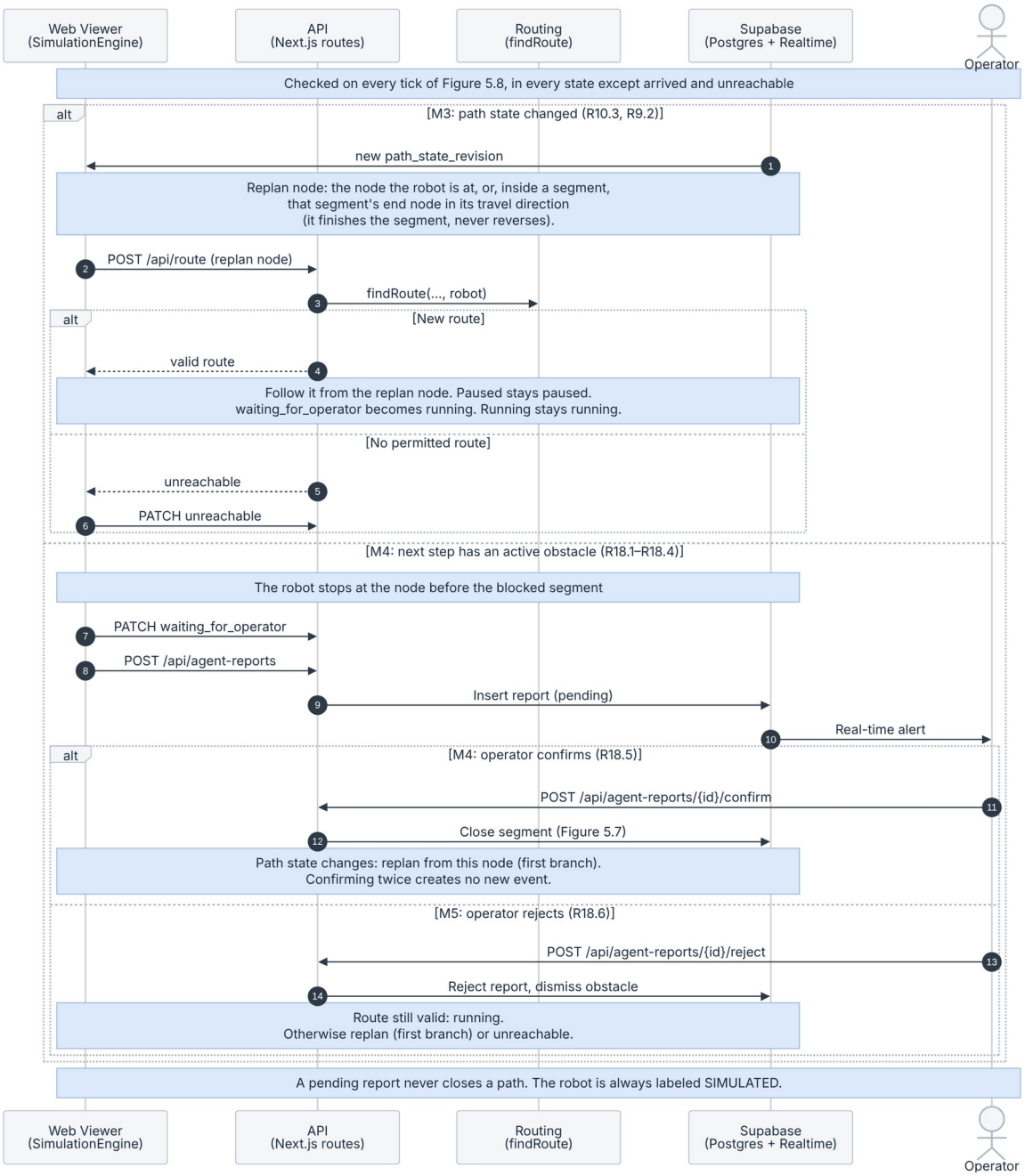


Figure 5.9: Simulated robot, path-change and obstacle events

What to notice

- Before entering each step, at the node, the robot checks that the step is still usable and has no active obstacle. Closures and obstacles are therefore caught before the robot moves into the affected segment.
- Events are handled in every state except arrived and unreachable. After a successful replan, a paused robot stays paused, a robot waiting for the operator's decision becomes running, and a running robot keeps running. No replacement route means unreachable.
- The replan starts at the node the robot is at. If the robot is inside a segment when it closes, it finishes that segment and replans from the segment's end node in its travel direction. It never reverses, so a one-way segment is never walked backwards, and it never skips a blocked segment.
- Pause, Resume and Reset are separate (M4). Resume first checks that the route is still valid at the current path_state_revision. Reset returns the robot to idle at its start with no route.

- Confirm (M4) closes the segment through Flow 4 and the robot replans. Reject (M5) dismisses the obstacle, and the robot resumes only if its route is still valid; otherwise it replans or becomes unreachable. The robot ends in arrived at the destination node.

Flow 6: Record a localization attempt and view test-point health

Purpose: A developer searches recorded attempts and, from M4, sees a health color for each test point.

Requirements: R15.1 (M2); R12.1, R12.2 (M3); R14.1–R14.5, R16.1 (M4).

Scope: [M2 partial] M2: search of labeled sample attempts. M3: attempts recorded by the phone (Flow 1). M4: health markers and the Gemini summary.

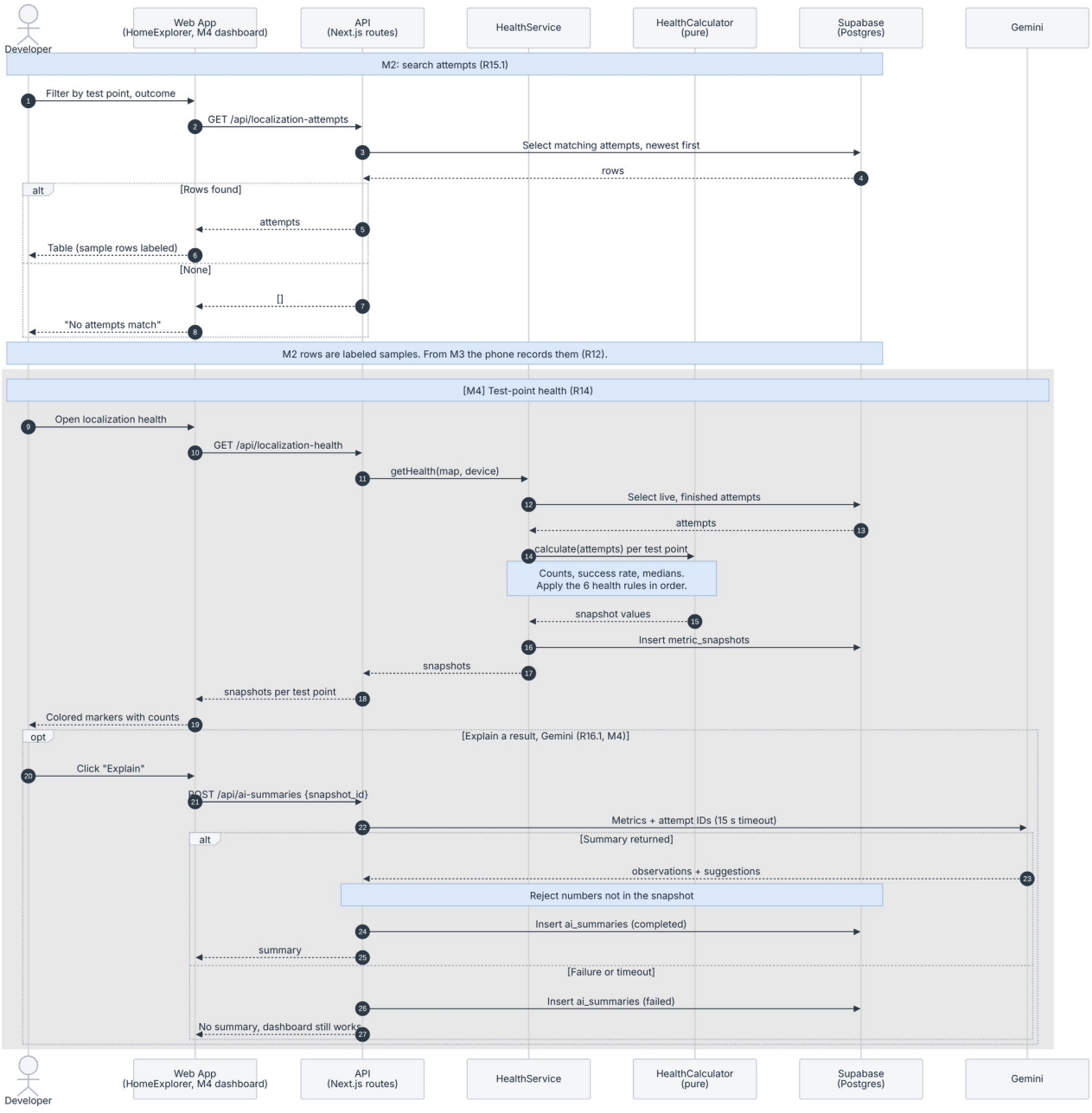


Figure 5.10: Attempt search and localization health

What to notice

- M2 search filters by test point and outcome, shows the newest first, labels sample rows, and shows “No attempts match” when nothing matches.

- Health uses only live, finished attempts from the same test point, device and map version; in_progress attempts are excluded. HealthService loads and saves; HealthCalculator only calculates.
- Health rules, in order, first match wins (Section 1.14): 1 gray, fewer than 10 attempts; 2 red, success below 60%; 3 red, at least 5 valid errors and median error above 1.5 m; 4 gray, fewer than 5 valid errors; 5 green, success 80% or more and median error 1 m or less; 6 yellow, all other cases. Counts are always shown with the color.
- This flow covers the colored markers (R14). The charts of R13 (M4) are not shown here.
- Gemini gets only aggregate metrics and attempt IDs. Numbers that do not match the snapshot are rejected, and the dashboard keeps working if Gemini fails or times out.

Flow 7: Stream the live phone position and follow the route

Purpose: A person walks with the iPhone while an observer watches /live. The dot moves on the map, the route starts from the node nearest the phone, and the page says when the person has arrived.

Requirements: R3.2, R3.3, R4.2, R6.2.1 (built early); R3.5, R6.5 (M2); R4.1 (partial); R6.2.2, R6.4 (M3).

Scope: [M2 partial] M2: pose upload, live dot, nearest-node start, route and arrival. M3: phone tracking states (R4.1), no route from a stale or lost pose (R6.4), start on the nearest path segment (R6.2.2).

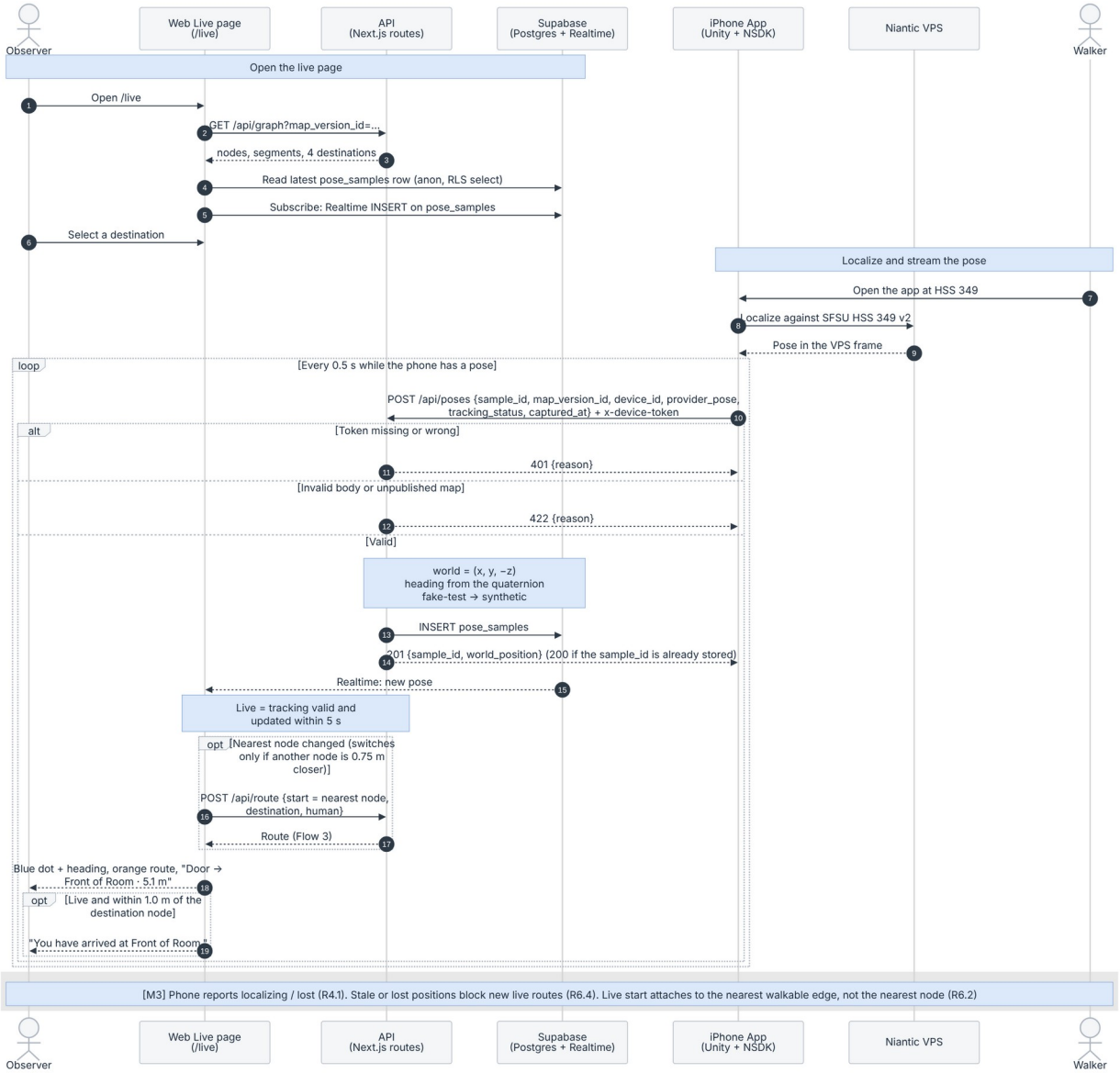


Figure 5.11: Live position, route and arrival

What to notice

- The phone sends the raw VPS pose every 0.5 s with a shared device token. The server flips Z into the world frame, computes the heading and stores both the raw and the converted pose in pose_samples. A repeated sample_id returns 200 and creates no duplicate.
- /live reads the latest pose once, then receives each new pose through Supabase Realtime. The position is live only when tracking is valid and the pose is less than 5 seconds old; otherwise the page shows “Not live” and the dot fades.
- The route start is the graph node nearest the phone (R6.2.1). It changes only when another node is at least 0.75 m closer, so the route does not flicker between two nodes. Each new start requests POST /api/route again (Flow 3).
- “You have arrived” appears only when the position is live and within 1.0 m of the destination node (R6.5). Choosing the same start and destination by hand shows “No route needed” instead, so old data never claims an arrival.
- Poses from the fake-test device (a pipeline test without localizing) are stored as synthetic and labeled “test pose” on /live (R3.5). In M2 they still move the dot and can start a route or trigger arrival; excluding synthetic poses from live navigation is planned for M3.
- In M2 the phone reports tracking as valid whenever it has a pose. Reporting localizing and lost (R4.1) and blocking routes from stale or lost positions (R6.4) follow in M3.

5.3 Traceability

Flow	Requirements	Main data classes	Built in M2	Later
1. Localize iPhone	R3.1; R3.4, R4.1, R12.1, R12.2	LocalizationAttempt, TestPoint, PoseSample, FrameTransform	Localize on the phone, show result (R3.1)	Attempt records, timeout, tracking states (M3)
2. View scanned area	R2.2, R2.3, R2.4, R15.3	MapVersion, VisualAsset, GraphNode, PathSegment, Destination	All	Active map from the site record, visual_assets table (M3)
3. Route	R5.1–R5.5, R6.1	Route, RouteStep, GraphNode, PathSegment, Destination	Manual-start route for a person or the robot, with distance	Revision check (409), stored routes (M3)
4. Close / reopen segment	R8.1–R8.3, R9.1, R9.3, R17.2	PathSegment, PathEvent, Route	Seeded availability only	Operator workflow (M3)
5. Simulated robot	R10.1–R10.5, R9.2, R18.1–R18.6	SimulatedAgent, SimulatedObstacle, AgentReport, PathEvent	Robot route profile only (Flow 3)	Start, move, replan (M3); pause, reset, obstacles (M4); reject (M5)
6. Attempts and health	R15.1; R12.1, R12.2; R14.1–R14.5, R16.1	LocalizationAttempt, MetricSnapshot, AiSummary	Search sample attempts (R15.1)	Recording (M3); health, AI summary (M4)
7. Live position	R3.2, R3.3, R3.5; R4.1, R4.2; R6.2.1, R6.2.2, R6.4, R6.5	PoseSample, GraphNode, Route	Pose upload, live dot, nearest-node start, route, arrival	Tracking states, stale/lost route block, segment attachment (M3)

The graph editor (R7.1–R7.2 in M4, R7.3 in M5) is not one of the seven flows. It follows the graph revision rule in decision 3 below.

5.4 Design decisions

1. **One routing engine for humans and robots.** Only the profile changes, so there is one algorithm to test. /viewer and /live both use it through POST /api/route.
2. **Optimistic concurrency using revision checks (M3).** Each change states the revision it expects, and so does each route request. The check and the write are one atomic conditional update, and a stale request gets 409. In M2 the route response already reports the revisions it used.
3. **Whole graph revisions.** A graph edit publishes a complete new graph revision (all nodes and segments) and switches current_graph_revision in one transaction. The old revision is kept, so stored routes and history still resolve. To be confirmed with the schema owner.
4. **Idempotent uploads that never go backwards.** One upsert with rules A–D (Flow 1): a missing record is created, retries change nothing, and a finished attempt never returns to in_progress. Failures are saved without a pose. Pose samples already work this way in M2: a repeated sample_id is not stored twice.
5. **Raw pose in, calculations on the server.** The phone sends the raw VPS pose. In M2 the server converts it and decides data_origin; from M3 it also calculates error and health.
6. **Pure logic separate from input and output.** Routing, pose origin and health rules have no database code, so they can be unit tested. Routing has 13 tests, and the full suite has 74.
7. **Check results before showing them.** Every route is validated against the database graph before it is sent, and arrival is shown only for a live, recent pose.
8. **One world frame per map.** For HSS 349 the world frame is the VPS map frame, so the scan, the graph nodes and the live poses line up without a stored transform. Other maps will use verified frame transforms (M3).
9. **AI explains numbers, never decides them.** The health color comes from fixed rules; Gemini output is checked against the snapshot and is optional.

5.5 Implementation status

Unit (Figures 5.3a, 5.3b)	Status on Oct 19, 2026	Where
RoutingEngine (findRoute), MinHeap	Merged into dev (PR #32). 13 Vitest tests pass.	app/frontend/lib/routing
GraphService, RouteService, LocalizationService (GET), SupabaseRepository, validation, routing adapter	Merged (PR #34) and deployed on Vercel.	app/frontend/app/api, app/frontend/lib/spatial/server
RoutePlanner, SceneViewer (/viewer)	Merged (PR #31). Real HSS 349 scan, database graph, POST /api/route and four destinations merged in PR #43 and released to main through PR #44 (Oct 19); deployed on Vercel and checked on the production URL.	app/frontend/components/viewer
HomeExplorer (home page list and search)	Merged (PR #39).	app/frontend/components/home
PoseService, LivePage (/live), PoseUploader	Merged (PR #35, #38, #40). Synthetic labeling of fake-test poses in PR #43, released through PR #44 and deployed (Oct 19).	app/frontend/app/api/poses, app/frontend/app/live, app/phone/live-position
iPhone app (Unity + NSDK)	Merged (PR #33). VPS localization in HSS 349 shown in class on Oct 13.	app/phone
Automated tests	74 tests pass: 61 API and logic tests, 13 routing tests.	npm test in app/frontend
All other units	Proposed design	—

PR numbers refer to github.com/ezerenm/SFSU-csc648-06-Fall26-team02.

6. Actual Key Risks

Each risk below is an actual issue in our current work, followed by how we will handle it.

6.1 Legal / content risks

1. Permission to publish the classroom scan has not been confirmed. HSS 349 is a shared campus room, and we have not yet confirmed that SFSU allows publishing its scan.

How we will handle this risk: We will confirm with our professor if we can use public scans for the demo. We will also scan only empty rooms and review the scans for any people, faces, or personal information.

2. Third-party licenses and plan limits. VPS runs on Niantic Spatial with the TA's pro plan, as well as Scaniverse, Three.js, Supabase and Vercel. Our repo is also Apache 2.0, so we should not have any SDK files that we can't share.

How we will handle this risk: We will review all of our third-party resources, their licenses and plan tiers from the README again to check for blockers. We will confirm that the app only references the Niantic SDK through Unity.

3. Live positioning and secrets. Supabase service keys should never be public. /live also shows the phone's position and the device ID without login. The pose data is currently public and the device token was shared.

How we will handle this risk: We will only keep the tokens in Vercel production variables and gitignored files. We will hide any device IDs from public view and, if needed, rotate the token if leaked. For pose data and the device token, in Milestone 3 we will restrict data reads and rotate the token after Milestone 2 for safety.

6.2 Technical risks

1. The live map lagged during the in-class demo (Oct 20). With the room full, the dot on /live updated unevenly and at times froze. Possible causes, not yet separated: tracking drops in a crowded room (the phone only sends while tracking is valid), the pose API running in a different region from the database, and one request per pose. Details and data: evidence/live-position.md.

How we will handle this risk: In M3, log tracking changes and failed posts on the phone, move the pose API next to the database (Vercel sfo1), measure end-to-end delay, and rehearse in a busy room with a recorded backup video before the next demo. Owners: Sumit (phone, API), Ezeren (deployment).

2. VPS reliability. Localization works in HSS 349, but how can we be sure it will be successful for the demo? Would different lighting or similar small changes affect VPS reliability?

How we will handle this risk: We will do additional testing at fixed test points in HSS 349 at different light levels and different times of the day.

3. Limited Mac and iPhone setups. Only Sumit and Daksh are testing with their phones and laptops.

How we will handle this risk: We will test whether the app can run on our other teammates' compatible devices.

4. Free services could stop. Vercel Hobby has build limits, Supabase free projects could pause when idle, and we are not sure how long we have the TA's Niantic Spatial pro subscription.

How we will handle this risk: We will test the live deployment and database connection a day before the demo, as well as one more check before presenting the demo in class.

5. The phone overlay has some Unity errors.

How we will handle this risk: Successful pose uploads were observed despite the errors; the cause remains under investigation and will be fixed in Milestone 3.

6. Scaniverse credits are shared with other teams because of the TA's pro plan.

How we will handle this risk: We will complete rescans early to make sure we have the resources available. We will also monitor the remaining credit balance before scanning.

6.3 Skill risks

1. Only two members can currently run the Unity VPS app.

How we will handle this risk: We will add the setup and execution steps in a guide inside the README. Additionally, we will do one more in-person test before M3.

2. There is no study plan for the stack (Next.js, Supabase, Three.js, Unity).

How we will handle this risk: In Milestone 3, each member will pick one to learn and share their notes and learnings in our Discord.

3. Only two team members know the 3D viewer code.

How we will handle this risk: We plan to have /viewer pull requests reviewed by other team members so more team members understand the code and structure.

4. Database migrations are run manually by 1–2 team members.

How we will handle this risk: We will keep every change as a migration file with steps, and we will store these files in the repository.

6.4 Schedule risks

1. We missed some internal deadlines after revisiting and changing the plan. Most of the work was completed in the last week.

How we will handle this risk: We will break the work into more digestible tasks and give updates about our sections through Discord. We will also review our progress in each meeting, in person and online.

2. Our prototype had an internal deadline of October 12, but it was mostly completed on October 16 and finished on October 19.

How we will handle this risk: Remaining fixes are planned and will be completed in Milestone 3.

6.5 Teamwork risks

1. Some documentation sections were written based on older code.

How we will handle this risk: Each team member will recheck their section against the latest code on main before submission.

2. Task cards weren't kept updated, so it was hard to determine if a team member was blocked.

How we will handle this risk: We will update the task cards every meeting and post blockers in Discord the same day.

6.6 Risks resolved or reduced during M2

- iPhone VPS localization at the testing site (SFSU): localization works and the position appears on /live.
- VPS processing of the testing classroom (HSS 349): the scan is now in use.
- Viewer using test / temporary data: /viewer now uses the real scan and the real database.
- Device token in Git: partially mitigated. A gitignore rule (PR #38) prevents committing the token file again, but this does not invalidate a token that was shared; the token will be rotated after M2 grading (see 6.1, risk 3).
- Conflicting branches: we have three clean dev → main releases.
- Phone → production upload: verified Oct 20 (828 pose samples stored across 8 upload periods, separated by gaps of more than 60 seconds; see evidence/live-position.md).

7. Project Management

Team meetings. Each meeting starts with a short check-in before the planned agenda. Every member reports what they have finished, what they are working on, and anything at risk of slipping, including tasks that may need more time. Because members are open about their workload and where they need help, the team can spot problems early. When needed, tasks are reassigned or another member is brought in to help. Day-to-day communication happens on Discord, and meeting notes are kept in a shared meetings document.

Task tracking. Individual work is tracked on a GitHub Projects board. Each member has a task guide card, created as a GitHub issue in the team repository and assigned to them. Each card lists the member's role, their required tasks with due dates, and a short how-to guide for each task. Members check off tasks as they finish them and move their card through the board's columns: To Do, In Progress, Blocked, In Review and Done. The board is reviewed at each meeting to confirm everyone is progressing, and blockers are raised the same day so they can be resolved quickly.

Code review. All code is developed on feature branches and merged into the dev branch through pull requests, each reviewed by another team member before merging. When a release is ready, dev is merged into main.

Lessons learned. Cards were not always updated promptly, and some deadlines slipped. For M3, we will break work into smaller weekly tasks so progress is easier to track and delays surface sooner.